WEEK 9
-
OBSERVABILITY

10 November 2025
11 November 2025
12 November 2025

Simone Maggiani

ABOUT ME

Name:

- Simone Maggiani

Education:

- Bachelor's Degree in Computer Engineering

Past Working Experiences:

- Senior Consultant in Capgemini (11+ years)

Current Working Experience:

- **Solution Architect** in MSC Technology (4+ years)
- Enterprise Architecture (EA) team

ENTERPRISE ARCHITECTURE (EA) TEAM

Standardization

Accelerated Delivery

Transversal Automation

Self-Service & DevOps

The Enterprise Architecture team ensures the standardization and governance of technology, software design, and systems integration in alignment with business strategy enabling scalable, secure, and product-driven operations while supporting innovation and long-term planning.

AGENDA

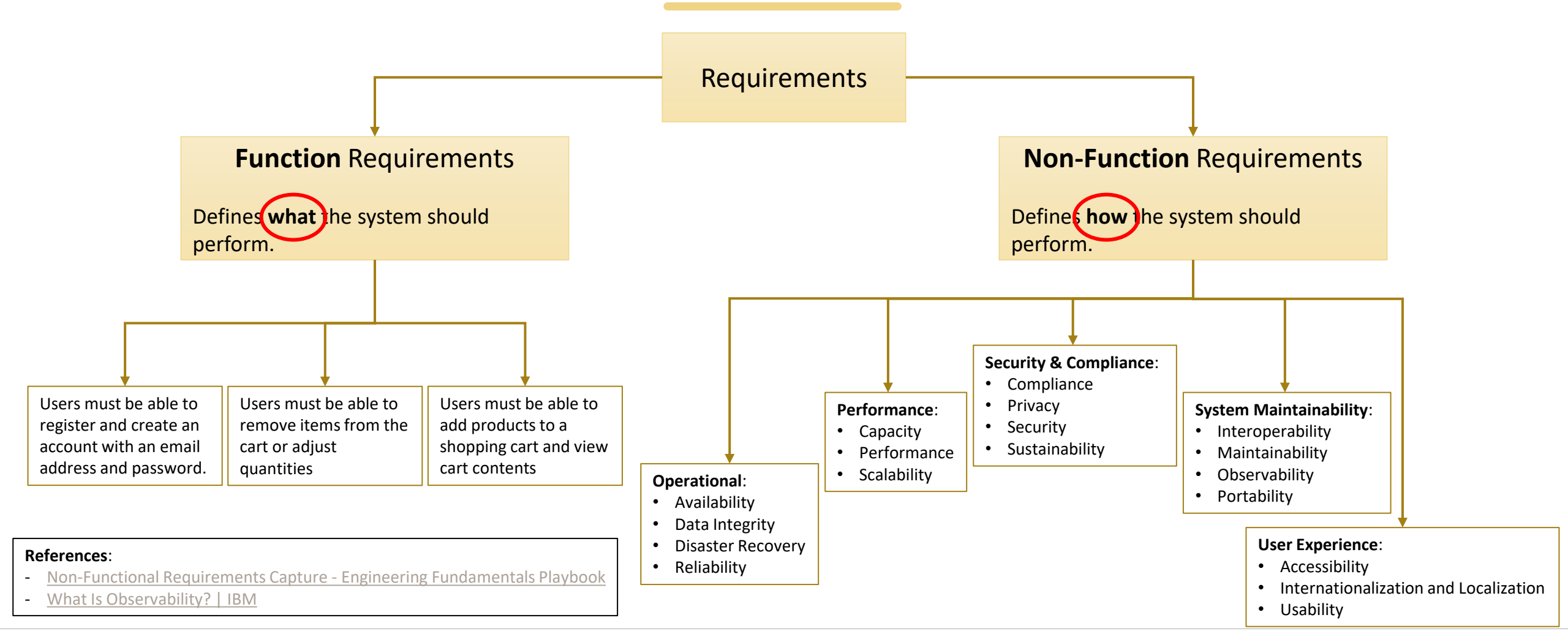
Theory:

- Functional Requirements VS Non-Functional Requirements
- Observability
  - Three pillars
    - Logs
    - Metrics
    - Traces
  - Example
  - Health Check
  - APM
  - Monitoring Tools
    - Application nsights
    - DataDog

Implementation:

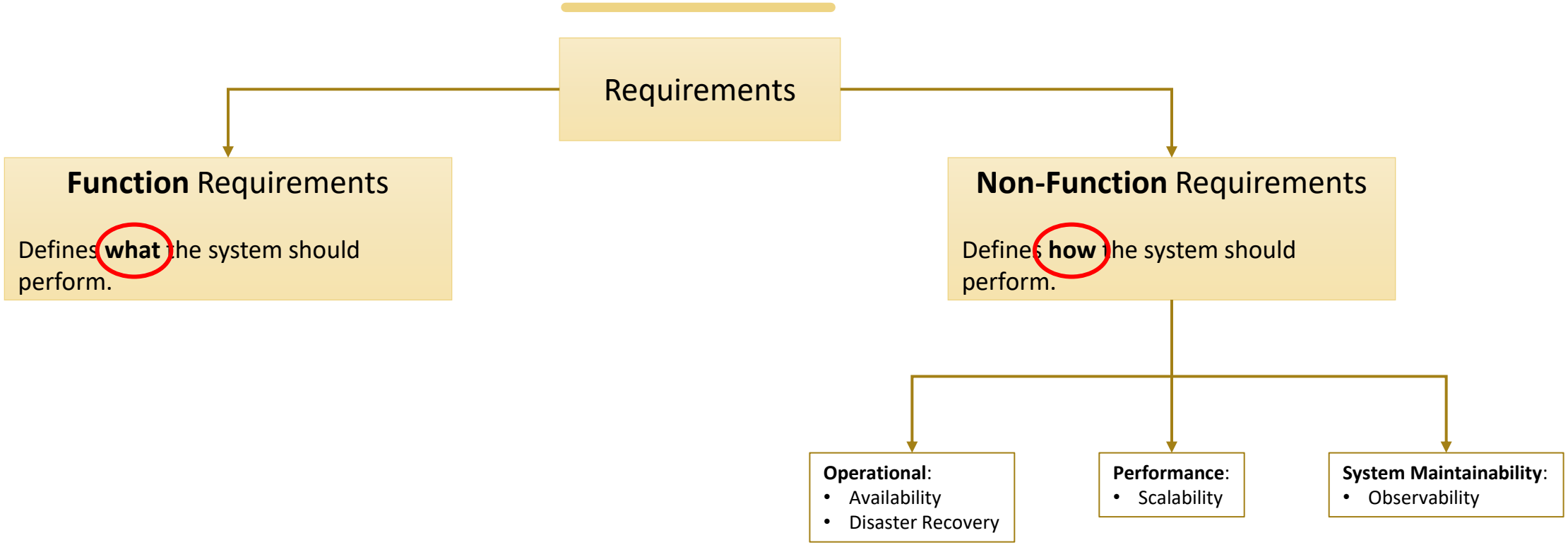
- Logs
  - General Concepts
  - Adding Log in Console App
  - Adding Log in ASP.NET
  - Configuring Log (via code or via configuration)
  - Structured Log
  - Serilog
  - Exercises
    - 1.1.Logging.Microsoft.SDK
    - 1.2.Logging.Microsoft.SDK.Structured
    - 1.3.Logging.Serilog
    - 1.4.Logging.Serilog.Datadog
- Metrics
- Traces
- Health Check
- Complete Exercise

REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL

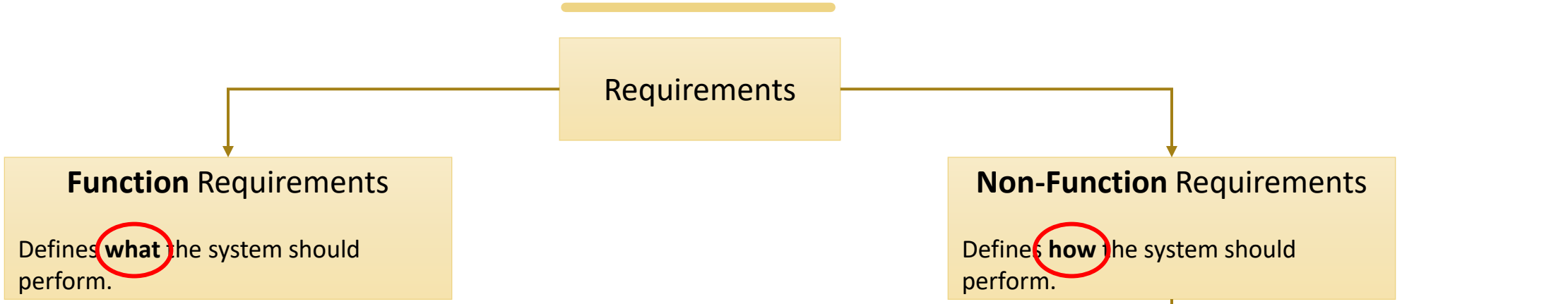


References:
- Non-Functional Requirements Capture - Engineering Fundamentals Playbook
- What Is Observability? | IBM

REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



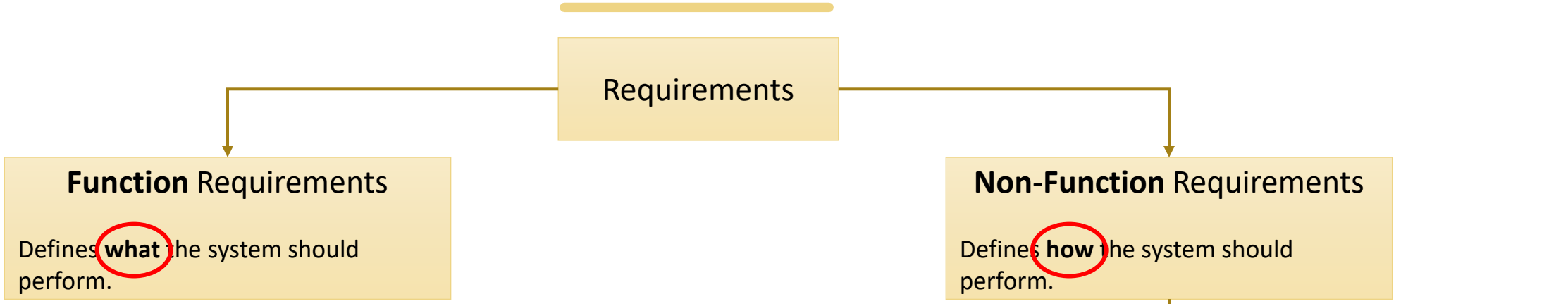
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Availability

- **Definition**:
  - System's uptime and accessibility to users
- **Example**:
  - The web portal must maintain 99.95% uptime per month, excluding planned maintenance periods

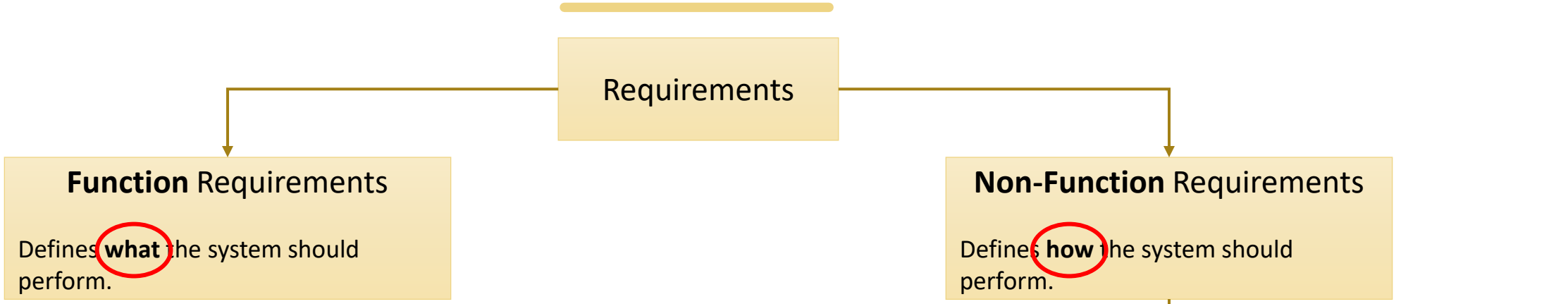
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Disaster Recovery

- **Definition**:
  - Defines how the system restores operations after a major failure or disaster (e.g., data center outage, cyberattack, or natural disaster). It focuses on recovery time and data loss limits.
- **Example**:
  - In case of a system failure, services must be restored within 4 hours (**RTO**), and no more than 15 minutes of data (**RPO**) may be lost.

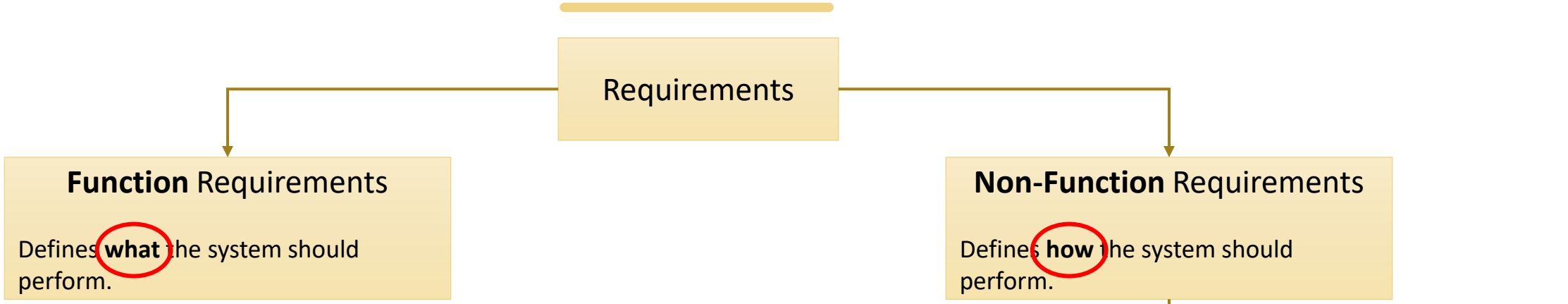
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Scalability

- **Definition**:
  - Defines the system's capacity to handle increasing workloads without performance degradation
- **Example**:
  - The system must support a **10× increase** in active users **with less than 20% performance impact**.

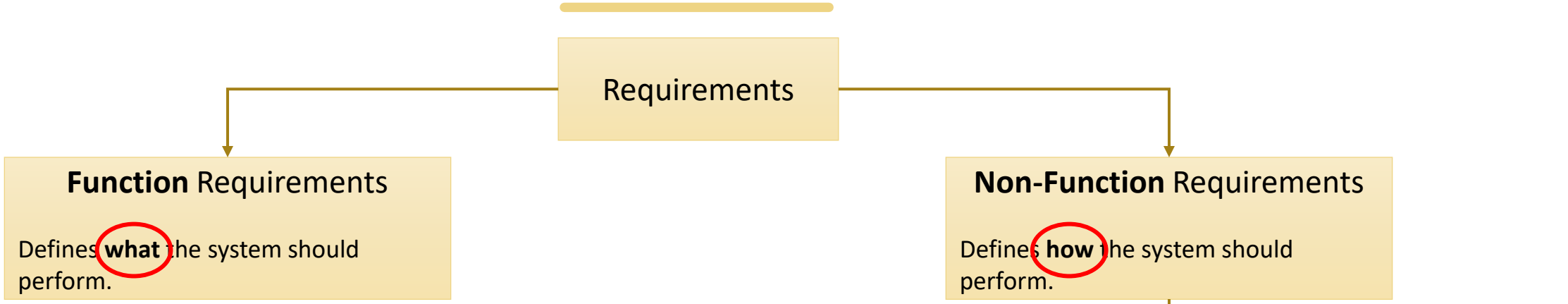
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Observability

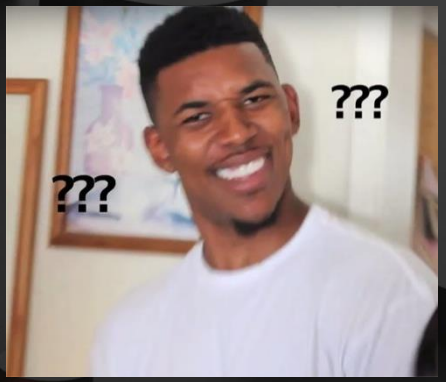
- **Definition**:
  - The ability to measure a **system's internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces
- **Example**:
  - The system must provide centralized logging, metrics, and distributed tracing that allow engineers to identify the root cause of incidents.

REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL

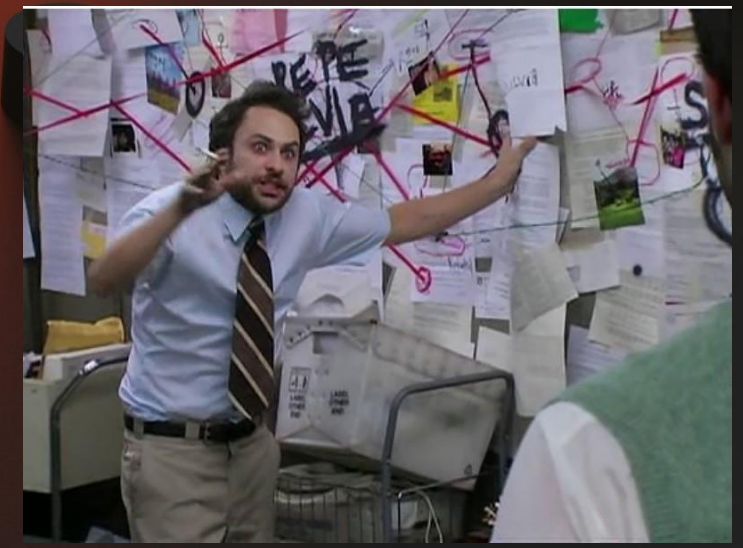


Observability

- **Definition**:
  - The ability to measure a **system's internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces
- **Example**:
  - The system must provide centralized logging, metrics, and distributed tracing that allow engineers to identify the root cause of incidents.



Q&A
Time



OBSERVABILITY

What:

- The ability to measure a system's **internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces.
- You can tell what's happening under the hood of the system without opening it.

Why:

- Provide **holistic view** of the **application health**.
- **Identify** and **diagnose failures** to get to the problem fast.

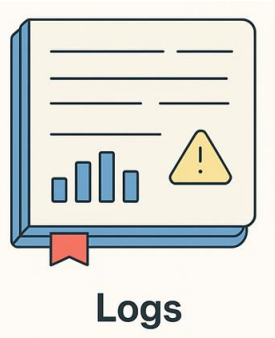
When:

- Always :)

PILLARS

Observability is composed of **three major pillars**:

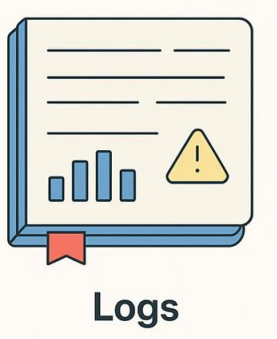
- **Logs**
  - **Logs** act as the system's **"captain's log"**, capturing significant events and what went wrong.



- **Metrics**
  - Metrics are **numerical measurements** collected at regular intervals.

- **Traces**
  - Traces record the **entire journey of an operation** as it moves across multiple services.

LOGS



What:

- A log is a detailed **record of events** that occur in the system, with timestamps for each one.

Why:

- To understand **what happened** and **why**
- Debugging & troubleshooting

When:

- When you need **context** around specific events, such as failures, exceptions, or unexpected behavior.
- Around key moments in your code

Example (file log "*logs.txt*"):

```
2025-11-03T14:12:45Z    INFO    AuthService     Login riuscito per utente: marco.rossi
2025-11-03T14:12:48Z    ERROR   AuthService     Login errato per utente: giuseppe.verdi
```

LOG LEVELS

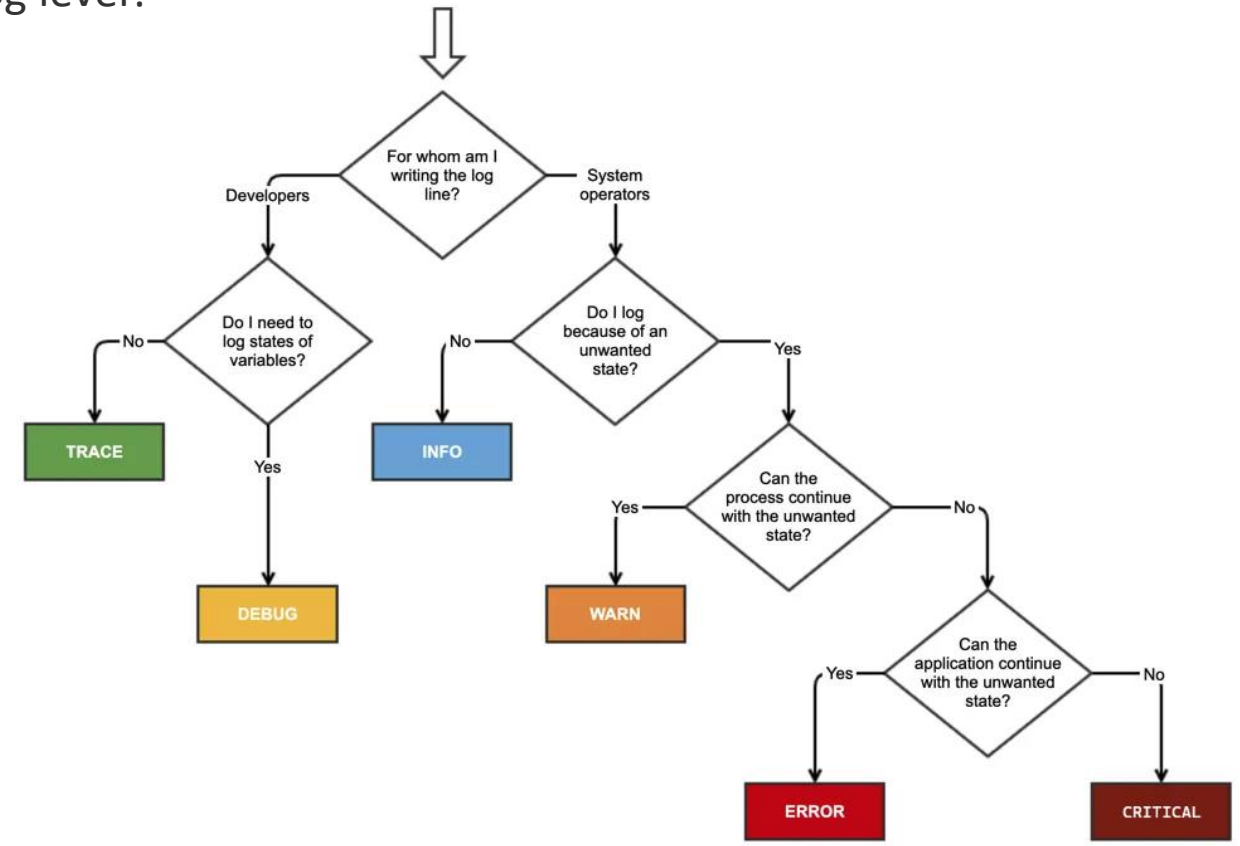
- Log levels are used to **categorize log events by severity** and control the verbosity of the logs

Name	Description
Trace	Logs that contain the most detailed messages. These messages may contain sensitive application data. These messages are disabled by default and should never be enabled in a production environment.
Debug	Logs that are used for interactive investigation during development. These logs should primarily contain information useful for debugging and have no long-term value.
Information	Logs that track the general flow of the application. These logs should have long-term value.
Warning	Logs that highlight an abnormal or unexpected event in the application flow, but do not otherwise cause the application execution to stop.
Error	Logs that highlight when the current flow of execution is stopped due to a failure. These should indicate a failure in the current activity, not an application-wide failure.
Critical	Logs that describe an unrecoverable application or system crash, or a catastrophic failure that requires immediate attention.
None	Not used for writing log messages. Specifies that a logging category should not write any messages.

LogLevel Enum (Microsoft.Extensions.Logging) | Microsoft Learn

LOG LEVELS

- How to choose the right log level?



Reference: Step-by-Step Guide to Logs in C#: Best Practices and Tips | by iamprovidence | Medium

STRUCTURED LOG

What:

- Structured Logs are **logs written in machine-readable format** (mostly in JSON), where the information is stored in **key-value pairs**.
- Structured logs are **queryable**.
- Example:

```
{
  "timestamp": "2025-11-03T14:12:48Z",
  "level": "ERROR",
  "service": "PaymentService",
  "event": "PaymentFailed",
  "userId": 12345,
  "orderId": 98765,
  "error": "Timeout",
  "gateway": "Stripe"
}
```

Why:

- If the logs are written as unstructured plain text, it's hard to query them and extract useful information.
- Example:
  - `2025-11-03T14:12:48Z ERROR PaymentService – Payment failed for user 12345, order 98765, timeout connecting to gateway`

STRUCTURED LOG

Unstructured logs:

```
2025-11-03T14:12:45Z INFO  PaymentService - Starting payment process for user 12345, order 98765
2025-11-03T14:12:46Z INFO  PaymentService - Sending payment request to gateway: Stripe
2025-11-03T14:12:48Z ERROR PaymentService - Payment failed for user 12345, order 98765, timeout connecting
to gateway
2025-11-03T14:13:12Z INFO  PaymentService - Starting payment process for user 67890, order 98766
2025-11-03T14:13:13Z INFO  PaymentService - Sending payment request to gateway: PayPal
2025-11-03T14:13:15Z INFO  PaymentService - Payment successful for user 67890, order 98766, transaction ID
TX-PAYPAL-456
2025-11-03T14:14:02Z INFO  PaymentService - Starting payment process for user 12345, order 98767
2025-11-03T14:14:05Z ERROR PaymentService - Payment failed for user 12345, order 98767, card declined
```

Example requirements: *"How many payments failed?"*

How can this requirement be fulfilled? Let's think it out together…

```
grep -E "ERROR PaymentService - Payment failed" logs.txt | wc -l
```

Find this hardcoded string (if it changes or is slightly different…)

File to search in

Command line utility to count

STRUCTURED LOG

Structured logs:

```
{ "timestamp": "2025-11-03T14:12:45Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentStart", "userId": 12345, "orderId": 98765 }
{ "timestamp": "2025-11-03T14:12:46Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentRequestSent", "gateway": "Stripe", "orderId": 98765 }
{ "timestamp": "2025-11-03T14:12:48Z", "level": "ERROR", "service": "PaymentService", "event": "PaymentFailed", "userId": 12345, "orderId": 98765, "error": "Timeout", "gateway": "Stripe" }
{ "timestamp": "2025-11-03T14:13:12Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentStart", "userId": 67890, "orderId": 98766 }
{ "timestamp": "2025-11-03T14:13:13Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentRequestSent", "gateway": "PayPal", "orderId": 98766 }
{ "timestamp": "2025-11-03T14:13:15Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentSuccess", "userId": 67890, "orderId": 98766, "transactionId": "TX-PAYPAL-456" }
{ "timestamp": "2025-11-03T14:14:02Z", "level": "INFO",  "service": "PaymentService", "event": "PaymentStart", "userId": 12345, "orderId": 98767 }
{ "timestamp": "2025-11-03T14:14:05Z", "level": "ERROR", "service": "PaymentService", "event": "PaymentFailed", "userId": 12345, "orderId": 98767, "error": "CardDeclined", "gateway": "Stripe" }
```

Example requirements: *"How many payments failed?"*

"How many payments failed?"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| summarize FailedPayments = count()
```

"How many payments failed today?"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| where timestamp between (startofday(now()) .. now())
| summarize FailedPayments = count()
```

"Failures Over Time (Trend binned by 1h)"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| summarize Failures = count() by bin(timestamp, 1h)
```

ACH:
To perform queries over logs, you must have a "Monitoring Tool"
(we'll see this later…)

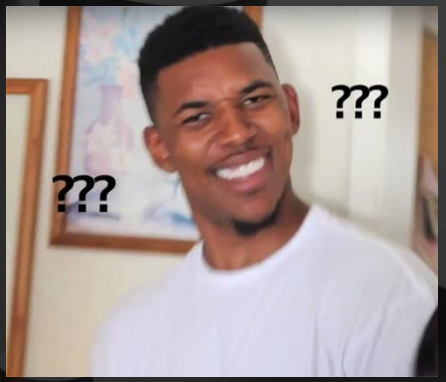
STRUCTURED LOG (COMPARISON)

Dimension	Unstructured Log	Structured Log
Format	Plain text	Key-value pairs (usually JSON)
Readability	**Easy** to read by **humans** **Hard** to read by **machines**	**Easy** to read by **machines** **Hard** to read by **humans**
Searchability	Low	High
Data Extraction	Low	High

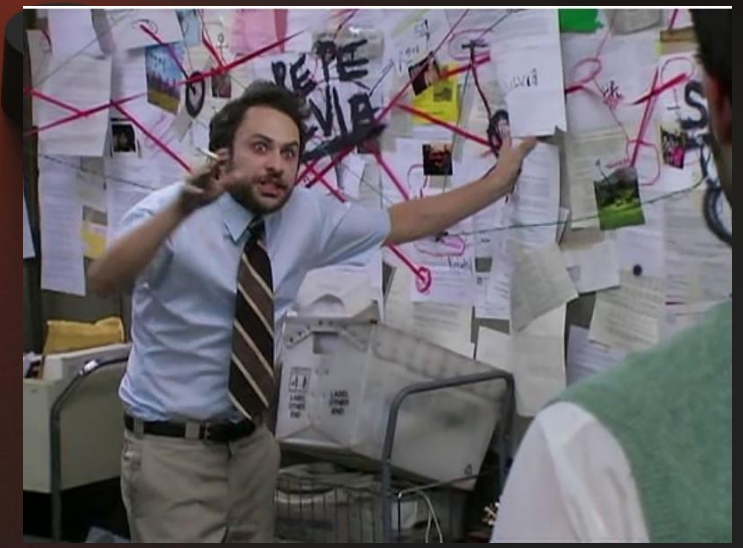
LOG TYPES

If even there is not official standard that defines log categories:

Log Type	Description	Typical Use	Who Uses it
Technical Logs (Application Logs)	**Events** generated **by the application code**	Debugging, troubleshooting, performance analysis.	Developers
System Logs	Events generated by the Operating System (OS) or the infrastructure	Monitoring **infrastructure** health, uptime, system errors.	IT Operations
Audit Logs	Record **who** (e.g., users) did **what**, **when**, and **how** in the system.	Compliance, traceability, accountability. E.g., my system must be compliant with an ISO standard.	Compliance officers
Security Logs	Authentication, authorization, intrusion detection, firewalls.	Security monitoring.	Security Team (SOC)
Business Logs	**Domain-specific actions** like order creation, invoice sent, shipment updated. These logs only **make sense within the context of the application domain**.	Business intelligence, user behavior, KPIs.	Product Owners / Business Analyst



Q&A
Time



METRICS

What:

- Metrics are **numerical measurements** collected at **regular intervals**.

Why:

- To **monitor trends** and **detect anomalies** over time

When:

- When you need to **track performance continuously** and **trigger alerts** on threshold.

METRICS

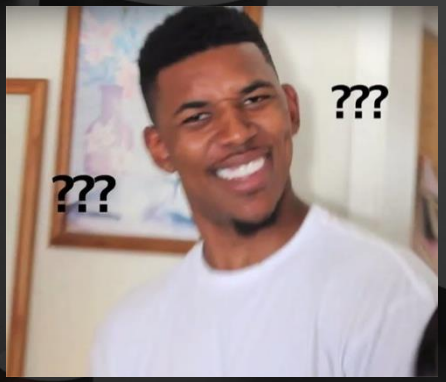
	Technical	
What		
Why		
When		
Example		

METRICS

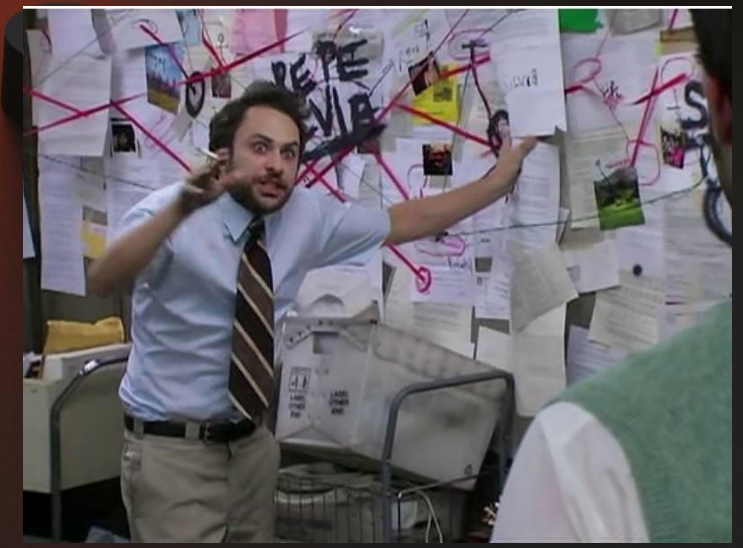
	Technical	
What	Numeric measurements collected at regular intervals to represent **system performance.**	
Why	To **monitor trends** and **detect anomalies** over time	
When	To **track performance continuously** and **trigger alerts** on threshold breaches	
Example	• API response time = 420 ms • Error rate (PaymentService) = 3.2 %	

METRICS

	Technical	Business
What	Numeric measurements collected at regular intervals to represent **system performance.**	Numeric measurements collected at regular intervals to represent **business outcomes**.
Why	To **monitor trends** and **detect anomalies** over time	To understand how the **system supports business goals**.
When	To **track performance continuously** and **trigger alerts** on threshold breaches	To extract **business outcomes** by **product owners and business analysts**.
Example	• API response time = 420 ms • Error rate (PaymentService) = 3.2 %	• Number of completed orders per hour = 250 • Cart abandonment rate = 8 %



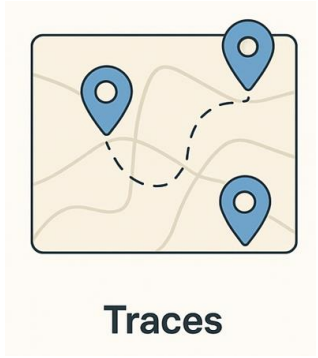
Q&A
Time



TRACES

What:

- A **trace** is the **entire journey of an operation** as it moves across multiple services.
  - An “operation” may refer to **a run for background job systems** (system always running that performs the same operations indefinitely)
  - An “operation” may refer to **a HTTP Request for Web API**



Traces

Why:

- To investigate the entire journey of the operation.

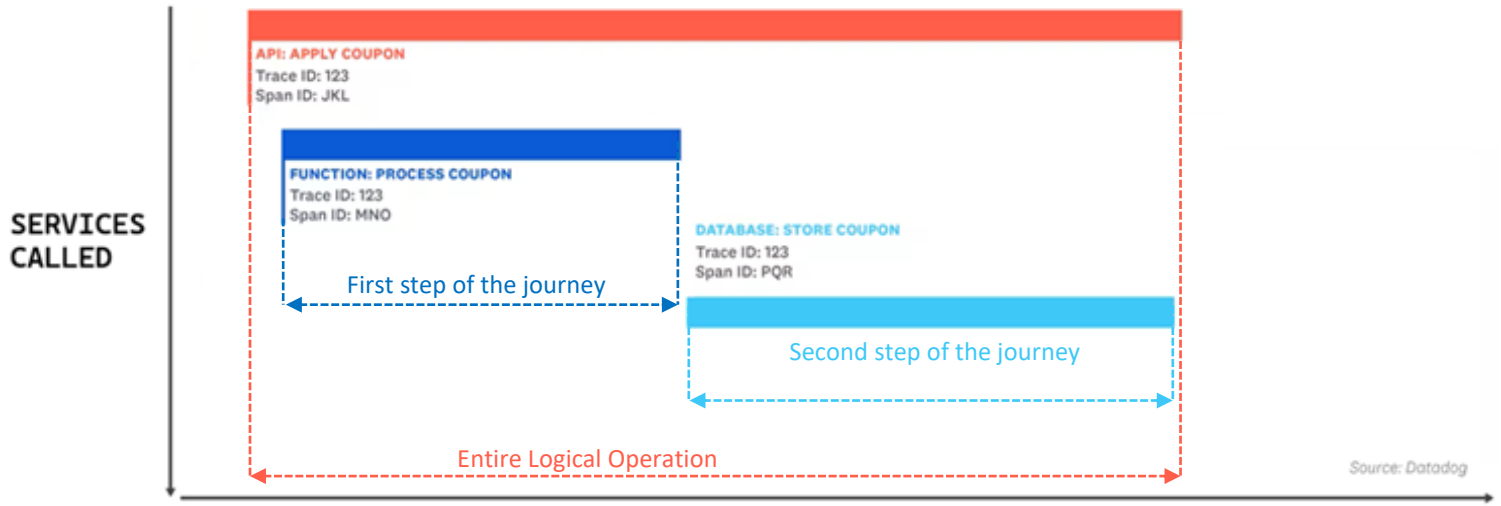
When:

- When you need to focus on **the entire end-to-end analysis of the operation**.

TRACES

Example:

- An e-shop Web API that apply a discount coupon
- To achieve this **Apply Coupon operation**, the system needs to:
  - **Verify the coupon** (e.g., well-formatted with regex)
  - **Store the coupon in the database** (to be not reusable)



TRACES - GLOSSARY

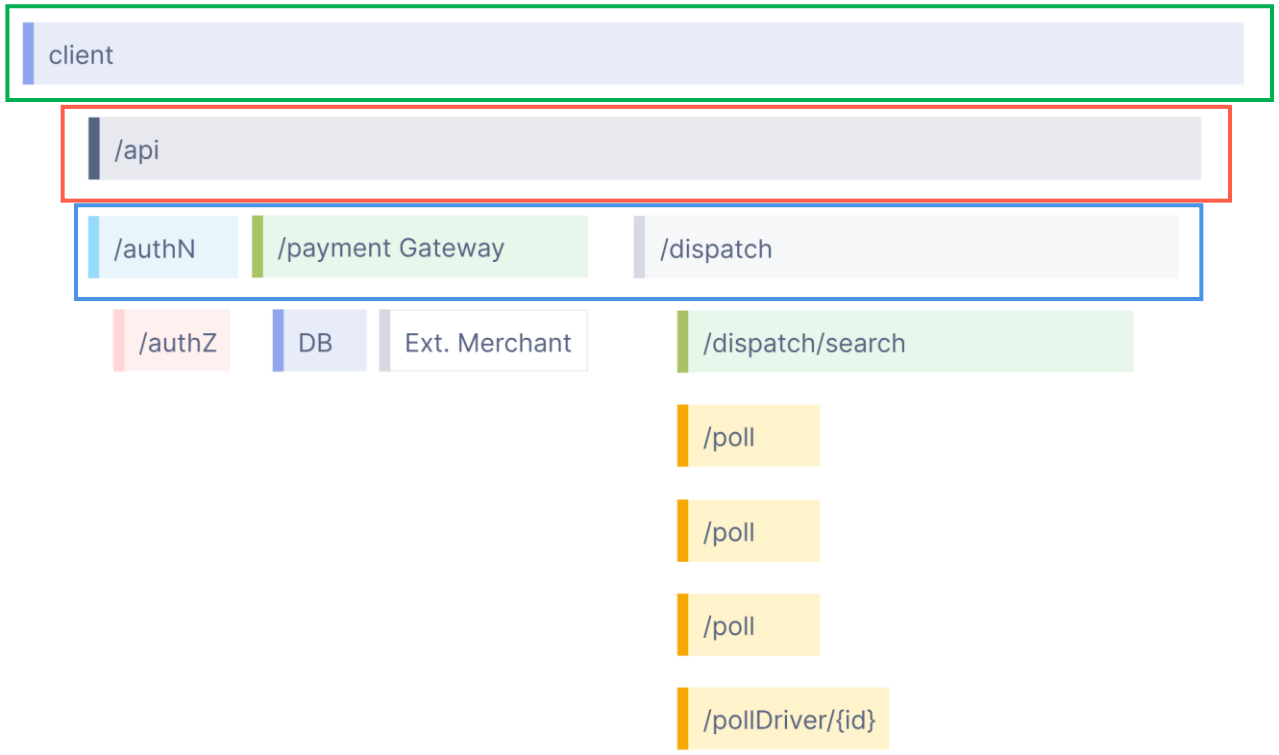
- **Trace**:
  - It represents the entire journey of an operation as it moves across multiple services.
  - A trace it's made up of **multiple spans.**

- **Span**:
  - A **single unit of work** within a trace. Such as a function call, database query, or external API call.
  - There is a parent-child relationship between spans:
    - **Parent Span**: the operation that **triggers** other work (e.g., a controller calling the database)
    - **Child Span**: the operation **triggered** by the parent (Database operation)

- **ID**:
  - **TraceId**: Unique identifier associated to the trace.
  - **SpanId**: Unique identifier associated to a single span within a trace.

- **Context Span**:
  - **Every span contains an object called Context**, that is **a bag of information to understand the position of the span inside the trace**
  - The Context object usually contains 3 main attributes:
    - **TraceId**: Identifies the overall trace and it's the same for all the spans inside a trace
    - **SpanId**: Identifies this specific span
    - **ParentSpanId**: Identifies the span that triggered this specific span

TRACES – HOW TO READ THE FLAME GRAPH

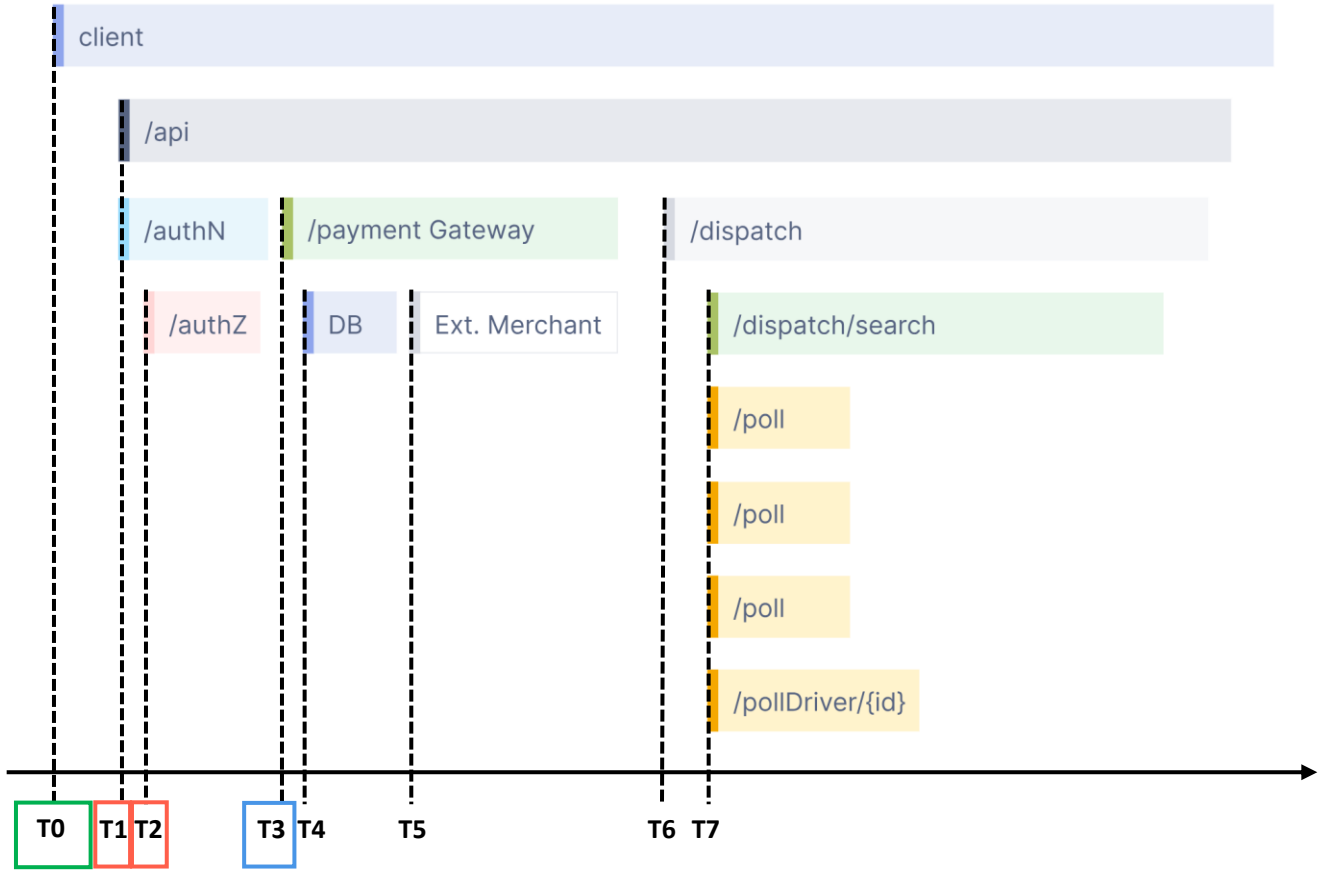


Observability primer | OpenTelemetry

Vertical axis (parent-child relationship):

- Flame Graph shows the parent-child relationship between spans.
- E.g.:
  - The first span that covers the entire operation is called "**client**"
  - The "**client**" span triggers the "**/api**" span
  - The "**/api**" span triggers:
    - "**/authN**" span
      - That triggers the "**/authZ**" span
    - "**/payment Gateway**"
      - That triggers the "**DB**" and "**Ext. Merchant**" spans
    - "**/dispatch**"

TRACES – HOW TO READ THE FLAME GRAPH



Horizontal axis (temporal time):

- It show the time when every span is executed
- E.g.:
  - **T0**: "**client**" span started
  - Simultaneously
    - T1: "**/api**" span started
    - T1: "**/authN**" span started
  - **T2**: "**/authZ**" span started
  - **T3**: "**/payment Gateway**" span started
  - **T4**: "**DB**" span started
  - **T5**: "**Ext. Merchant**" span started
  - **T6**: "**/dispatch**" span started
  - Simultaneously
    - **T7**: "**/dispatch/search**" span started
    - **T7**: "**/poll**" span started
    - **T7**: "**/poll**" span started
    - **T7**: "**/poll**" span started
    - **T7**: "**/pollDriver/{Id}**" span started

DISTRIBUTED TRACING

Distributed system:

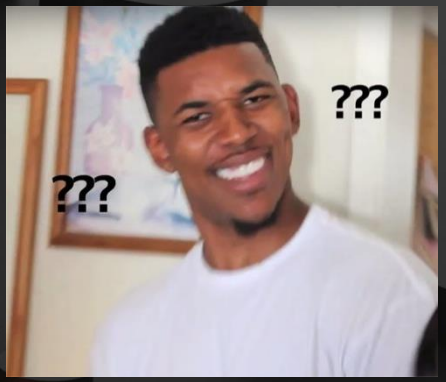
- A distributed system is a **collection of independent components**, often running on different machines, **that** communicate and **coordinate with each other to accomplish a single task**.

Example:

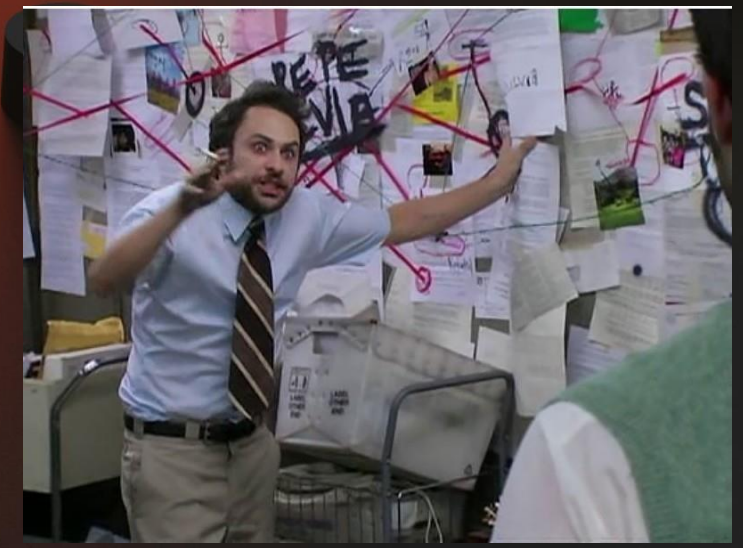
- When a customer places an order in an e-commerce site.
  - The **Frontend service** receives the request and validates the input.
  - The **Order service** creates the order record in the **database**.
  - The **Payment service** processes the payment and updates the **database**.
  - The **Email service** sends the confirmation.

Distributed Tracing:

- **Distributed Tracing = Tracing in Distributed Systems**
- Distributed tracing is the **entire journey of an operation** as it moves across **multiple services of a distributed system**.



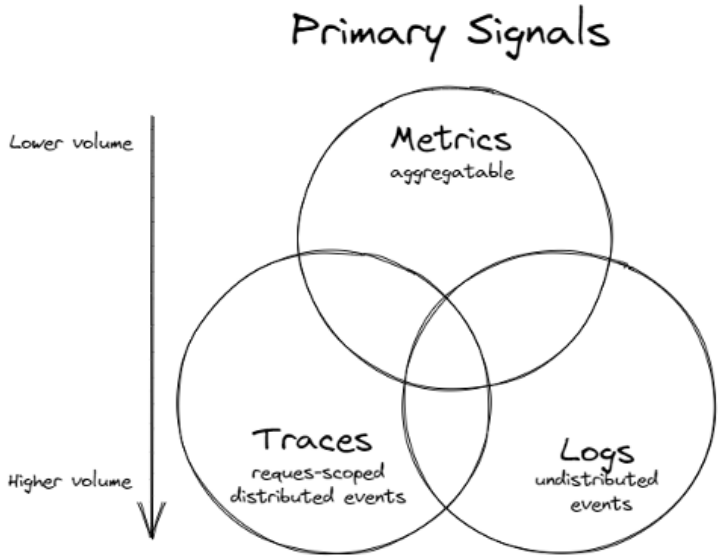
Q&A
Time



ALL TOGETHER: LOGS + METRICS + TRACES

When to use the right tool:

- **Logs**:
  - Use logs to **track detailed information about an event**, particularly errors, warnings or other exceptional situations.
- **Metrics**:
  - Use metrics to track the occurrence of an event, counting of items.
- **Traces**:
  - A trace provides visibility into how a request is processed across multiple services.



Logs vs Metrics vs Traces - Engineering Fundamentals Playbook

ALL TOGETHER: LOGS + METRICS + TRACES

Let's see the three pillars in a practical scenario.

- **Scenario Description**:
  - The process of buying an item on an e-shop system involves several components:
    1. **Frontend Service**: receives the request and validates order data
    2. **Order Service:** creates the order in the database
    3. **Payment Service**: calls an external provider to process the payment
    4. **Notification Service**: sends the confirmation email

  - Within a day, **multiple orders** are processed, **some succeed**, **some fail** due to different issues.

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService  User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService     Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService   Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService   Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService     Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService      Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService  User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService     Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService   Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService   Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService     Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService      Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService  User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService     Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService   Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService   Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService     Updating order 98767 status to PaymentFailed
```

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService        Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService        Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService       Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService       Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService        Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService        Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService       Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService       Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService        Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService        Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService        Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService        Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```

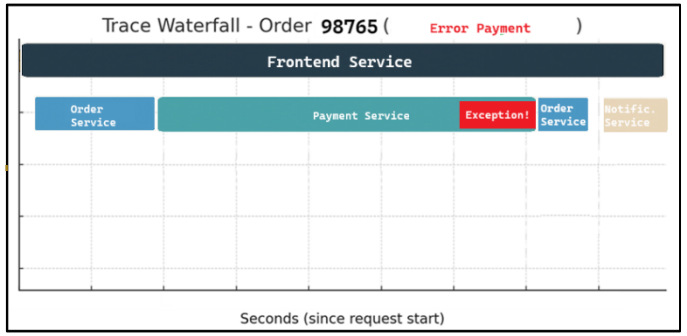
Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

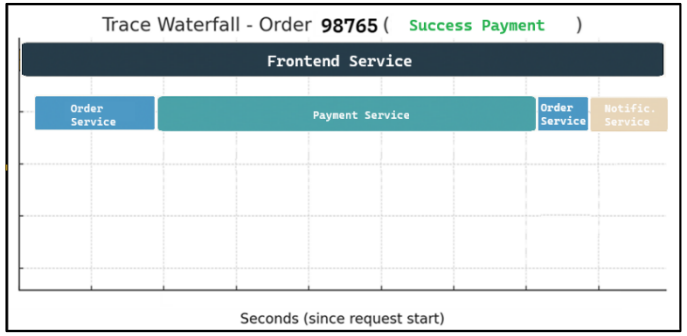
ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

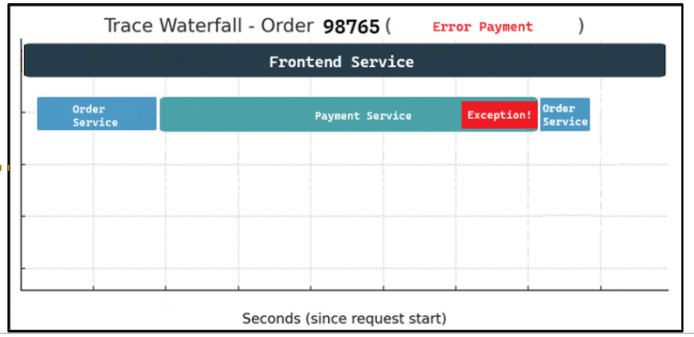
```
2025-11-03T09:00:00Z INFO  FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO  OrderService    Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService  Payment failed for order 98765, user 12345, timeout
                                           connecting to gateway
2025-11-03T09:00:05Z INFO  OrderService    Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO  NotificationService       Sending payment failure email to user 12345
```

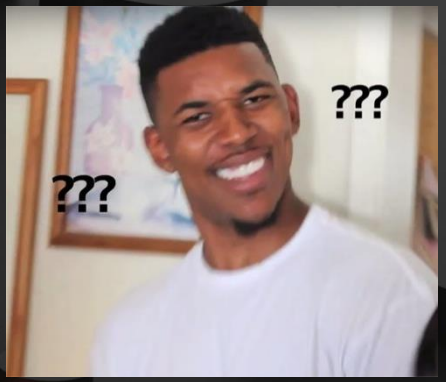



```
2025-11-03T09:15:12Z INFO  FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO  OrderService    Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO  PaymentService  Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO  PaymentService  Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO  OrderService    Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO  NotificationService       Sending confirmation email to user 67890
```

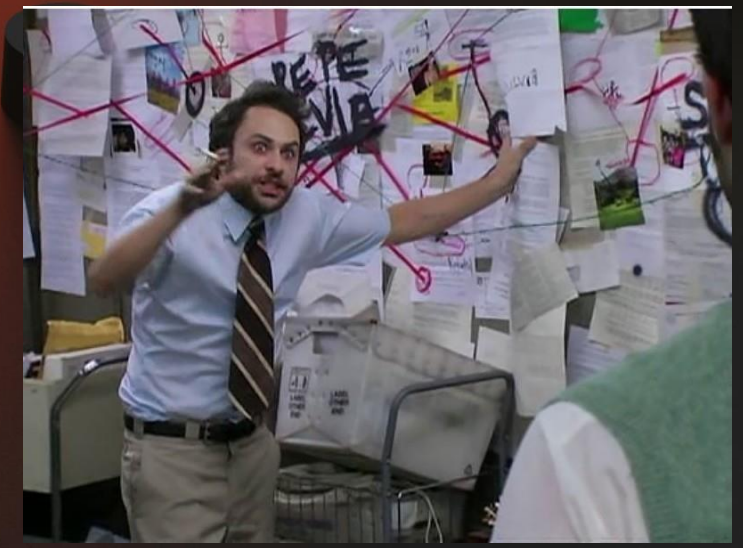



```
2025-11-03T10:42:33Z INFO  FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO  OrderService    Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO  PaymentService  Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService  Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO  OrderService    Updating order 98767 status to PaymentFailed
```





Q&A
Time



HEALTH CHECK

What:

- **A Health Check is a mechanism** that allows external systems (like monitoring platform) **to verify whether an application is healthy**.

Why:

- To **automatically detect service failures** before users notice and help platforms decide when to **restart unhealthy applications**.

When:

- Every time you need to detect failures before users notice
  - Lower environments: Recommended
  - Pre-Production: Mandatory
  - Production: Mandatory

HEALTH CHECK – ASP.NET

- **ASP.NET implementation**:
  - In **ASP.NET**, a health check is usually implemented as a **dedicated HTTP endpoint** that returns the status of the app.

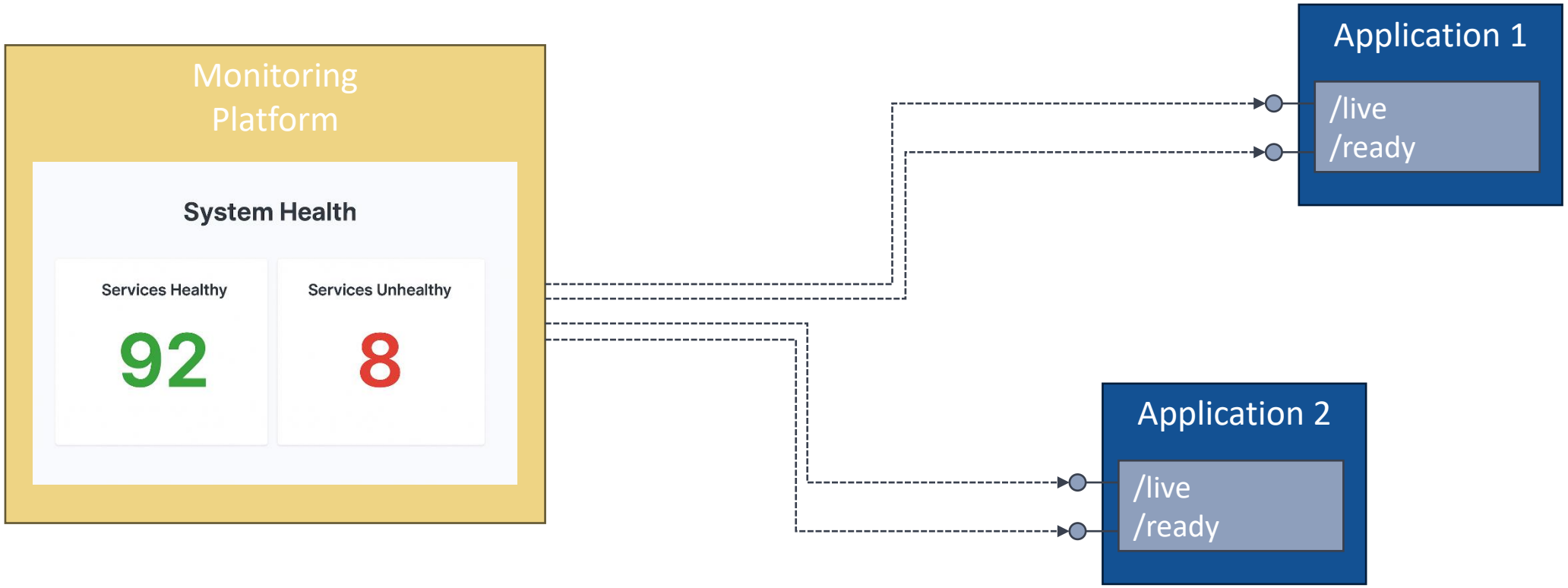
- There are two different endpoints for the health check:
  - "**/live**" endpoint (**Liveness**)
    - Purpose: ***Is the app alive?***
    - Example: Check if the app is running and not crashed.

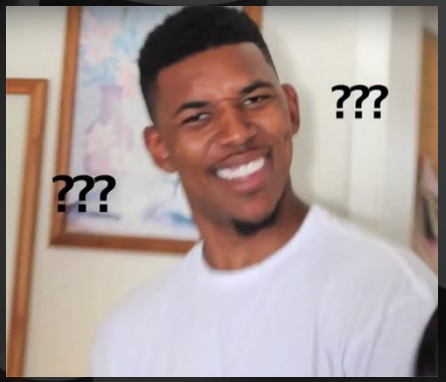
  - "**/ready**" endpoint (**Readiness**)
    - Purpose: ***Is the app ready to handle requests?***
    - Example: Check if the dependencies that my application calls are running. For example, the SQL Database used by my application.

HEALTH CHECK – MONITORING TOOLS

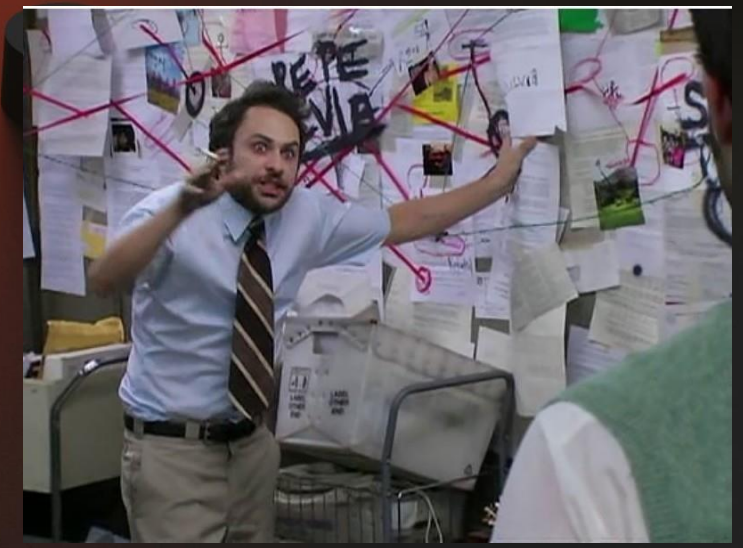
Monitoring tools:

- Monitoring tools regularly run health checks and display on dashboards whether each service is running or not.





Q&A
Time



APM – APPLICATION PERFORMANCE MONITORING

What:

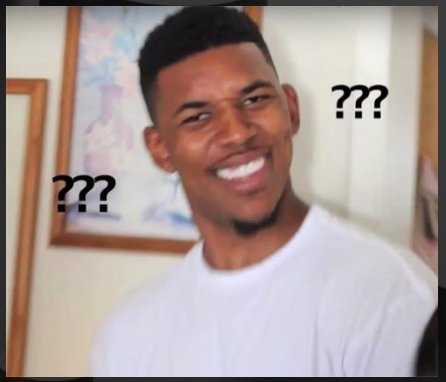
- APM **is** a solution to tracking, analyzing, and optimizing the performance and availability of software applications using telemetry data (metrics, logs, and traces).
- APM **is not** another “pillar” like metrics, logs, or traces, **it uses all three pillars** to understand, monitor and **improve application performance**.

Why:

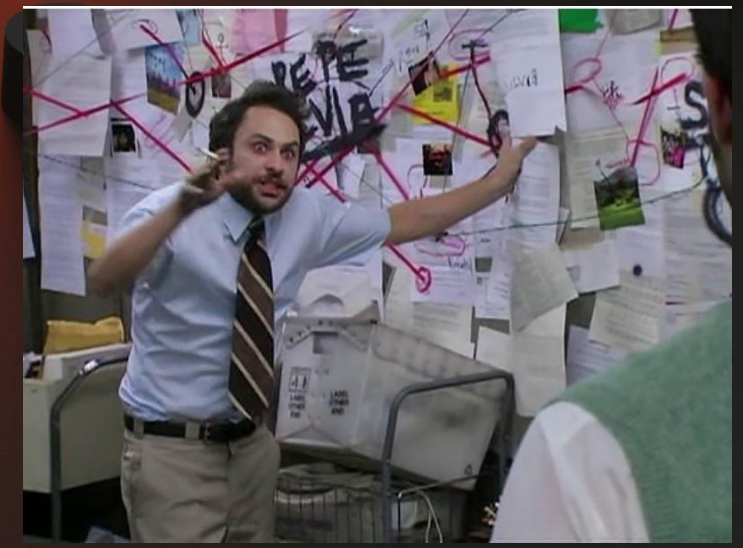
- To identify performance issue.

When:

- Every time you need to ensure the performance of the application.
  - Lower environments: Recommended
  - Pre-Production: Mandatory
  - Production: Mandatory



Q&A
Time



MONITORING TOOLS

What:

- **Monitoring tools** are platforms that **continuously** collect, analyze, and visualize data about your applications (and infrastructure).
    - The monitoring of the application is done using the three pillars: logs, metrics and traces.

Why:

- To improve reliability, optimize performance, and provide visibility into the entire system.

When:

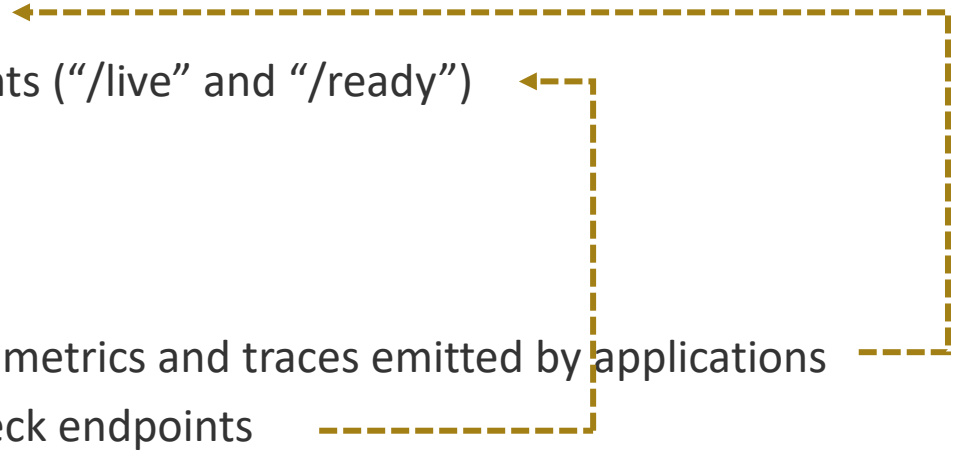
- Every time it’s needed to monitoring the system:
    - Lower environments: Recommended
    - Pre-Production: Mandatory
    - Production: Mandatory

To sum up…

MONITORING TOOLS

Applications:

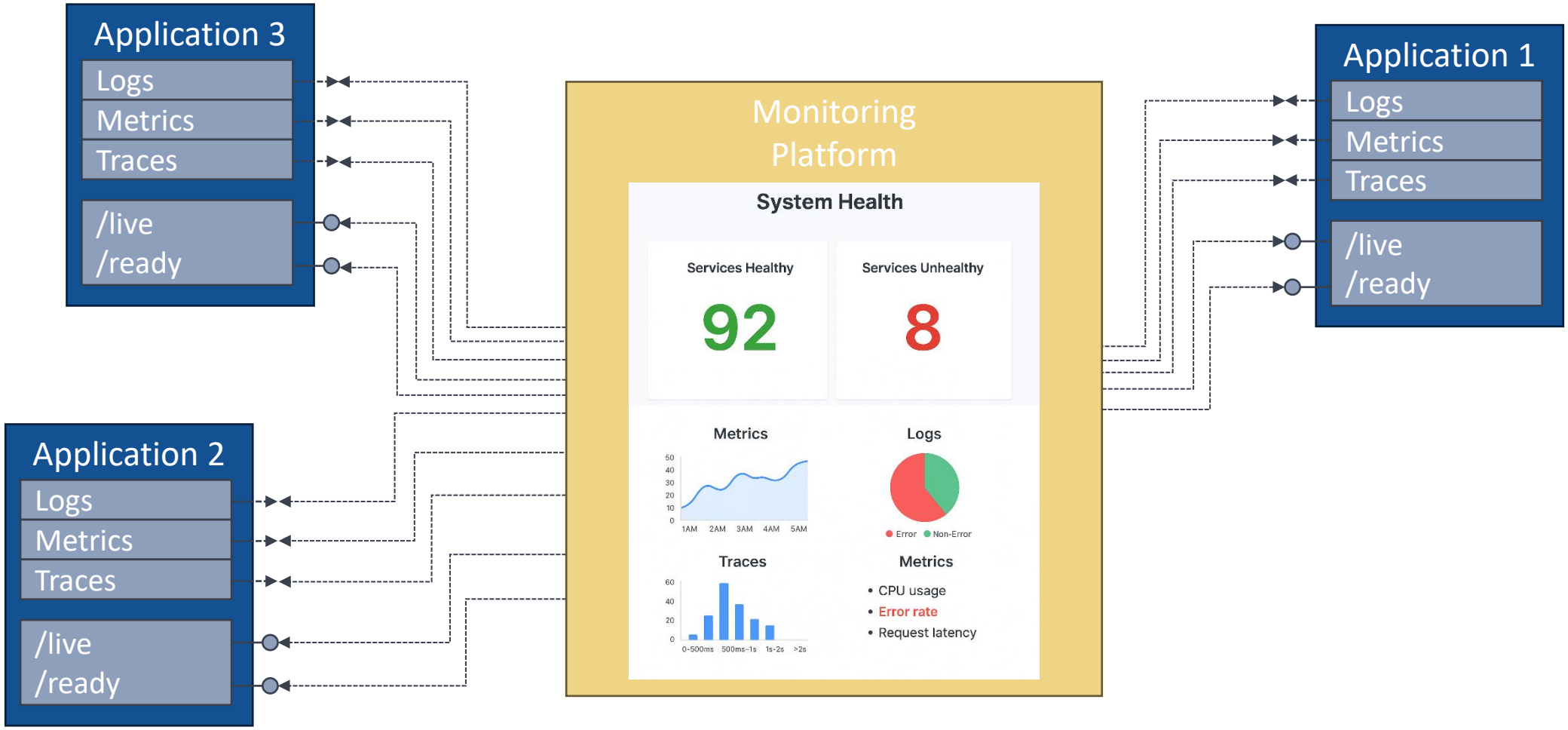
- Emit logs, metrics and traces
- Expose Health Check endpoints (“/live” and “/ready”)



Monitoring Tool:

- Continuously intercepts logs, metrics and traces emitted by applications
- Continuously calls Health Check endpoints
- **Summarizes and displays** the results through dashboards (charts) to provide **a holistic view of the application**.

MONITORING TOOLS



MONITORING TOOLS

Application Insight

- Provided by Microsoft Azure
- Deep integration with Azure ecosystem
- No longer the official platform
  - Used by systems that have not yet migrated
  - Used by systems that cannot migrate (technical constraints)

MONITORING TOOLS

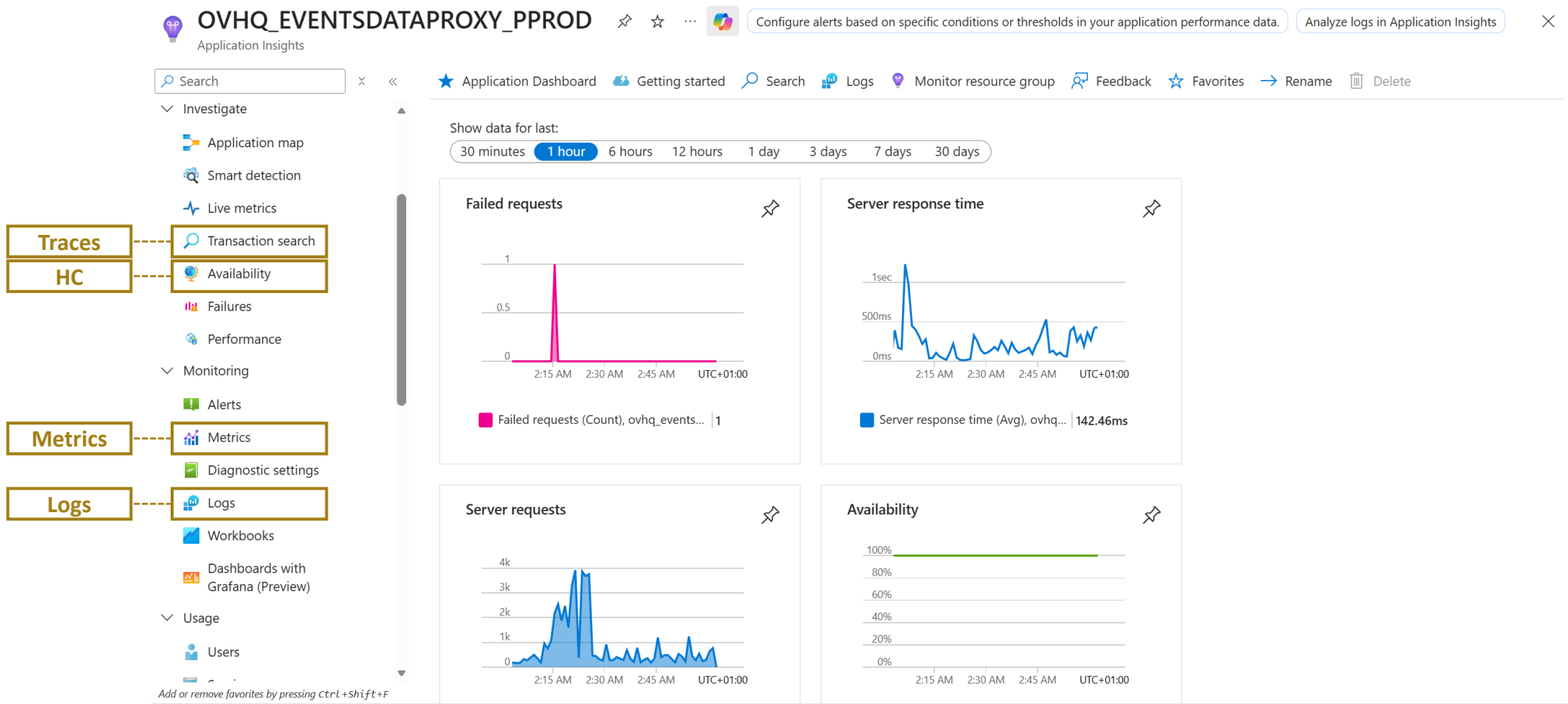
Application Insight

- Provided by Microsoft Azure
- Deep integration with Azure ecosystem
- No longer the official platform
  - Used by systems that have not yet migrated
  - Used by systems that cannot migrate (technical constraints)

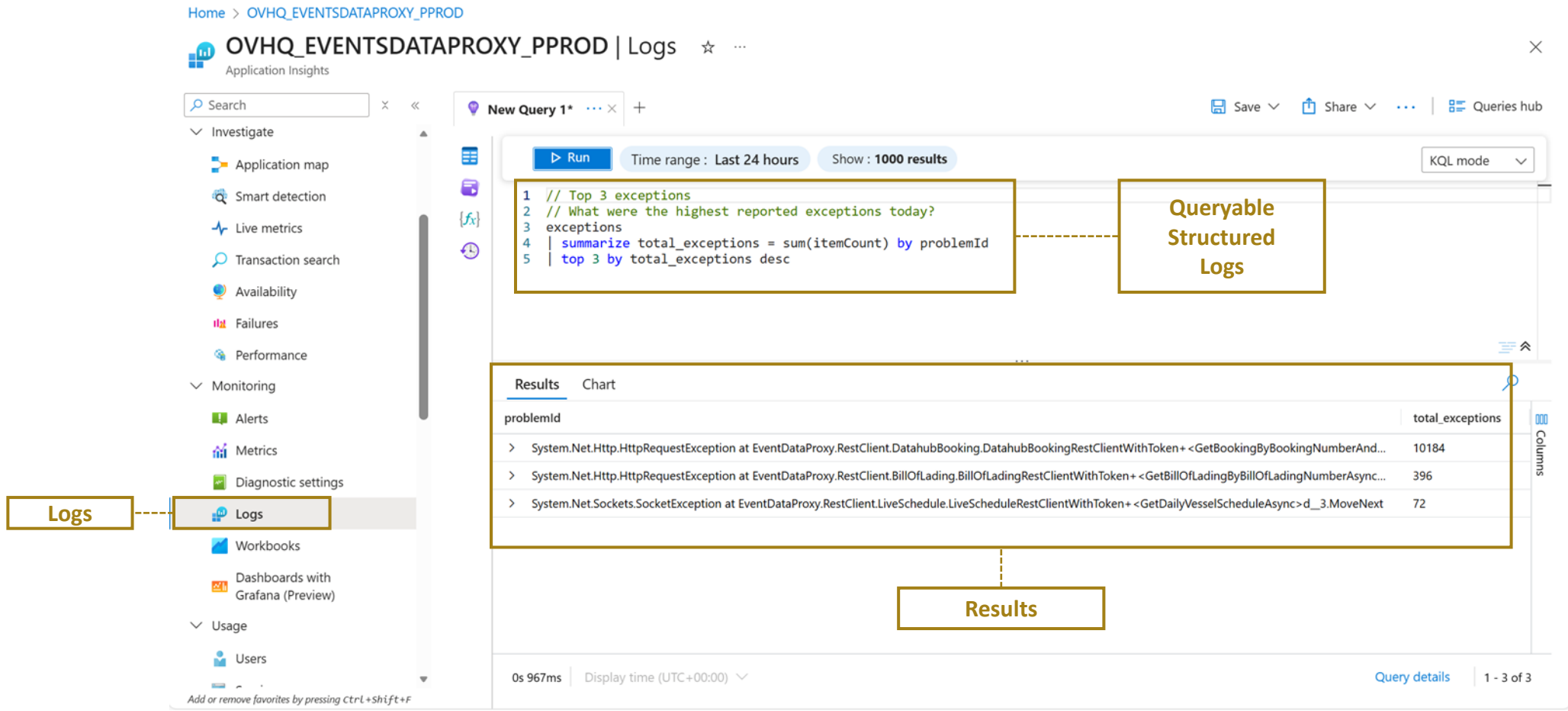
DataDog

- Provided by Datadog, Inc.
- Multi-cloud and platform-agnostic
- **Official platform in MSC Technology**

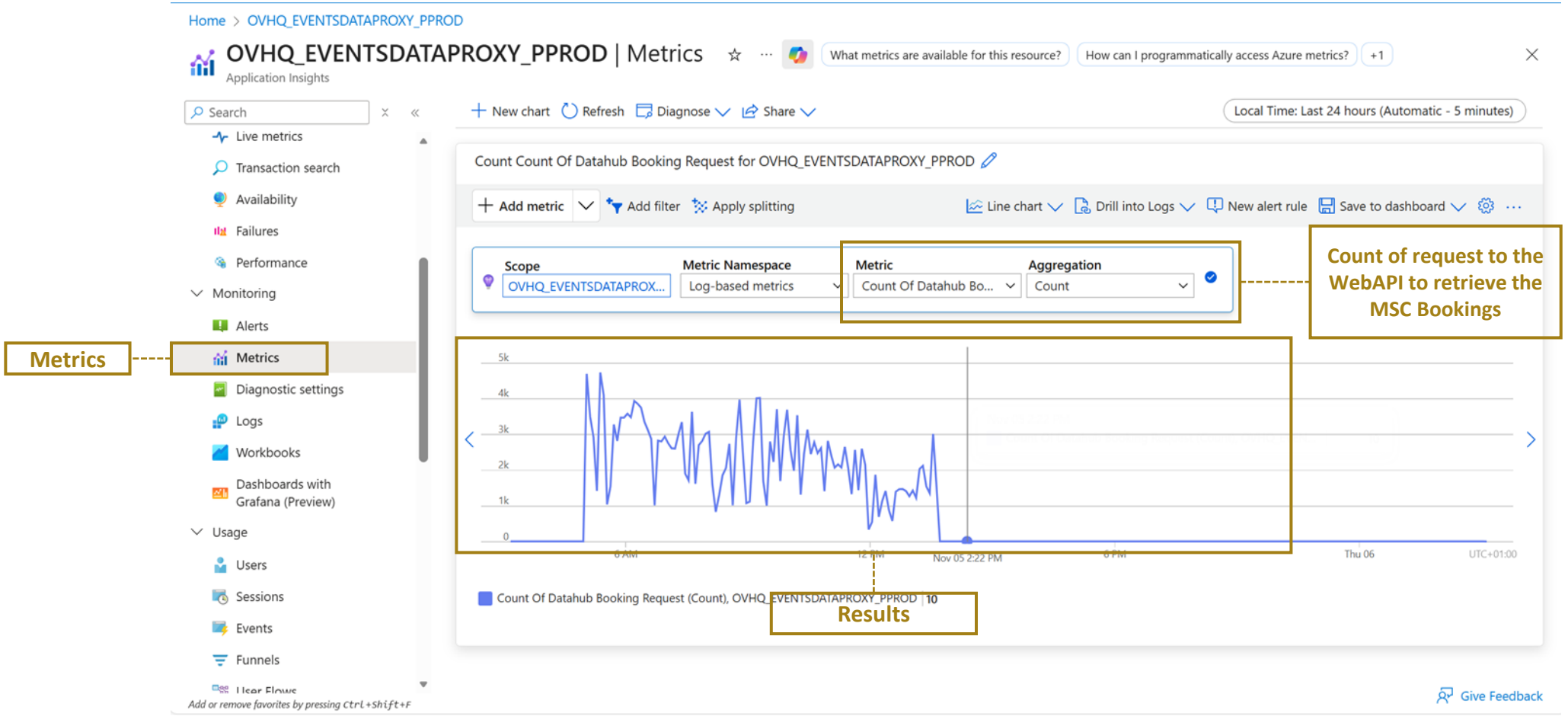
APPLICATION INSIGHTS - HOMEPAGE



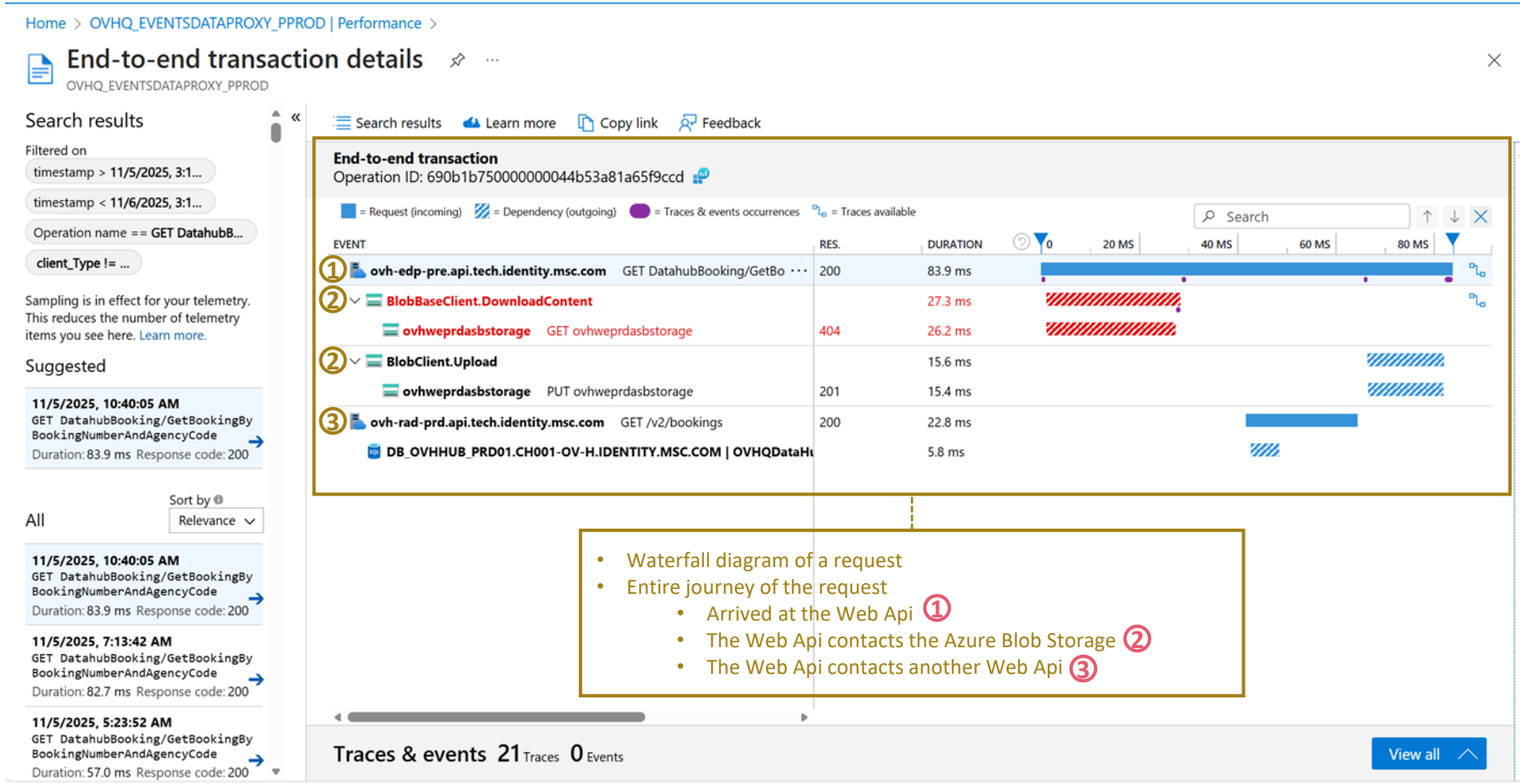
APPLICATION INSIGHTS - HOMEPAGE



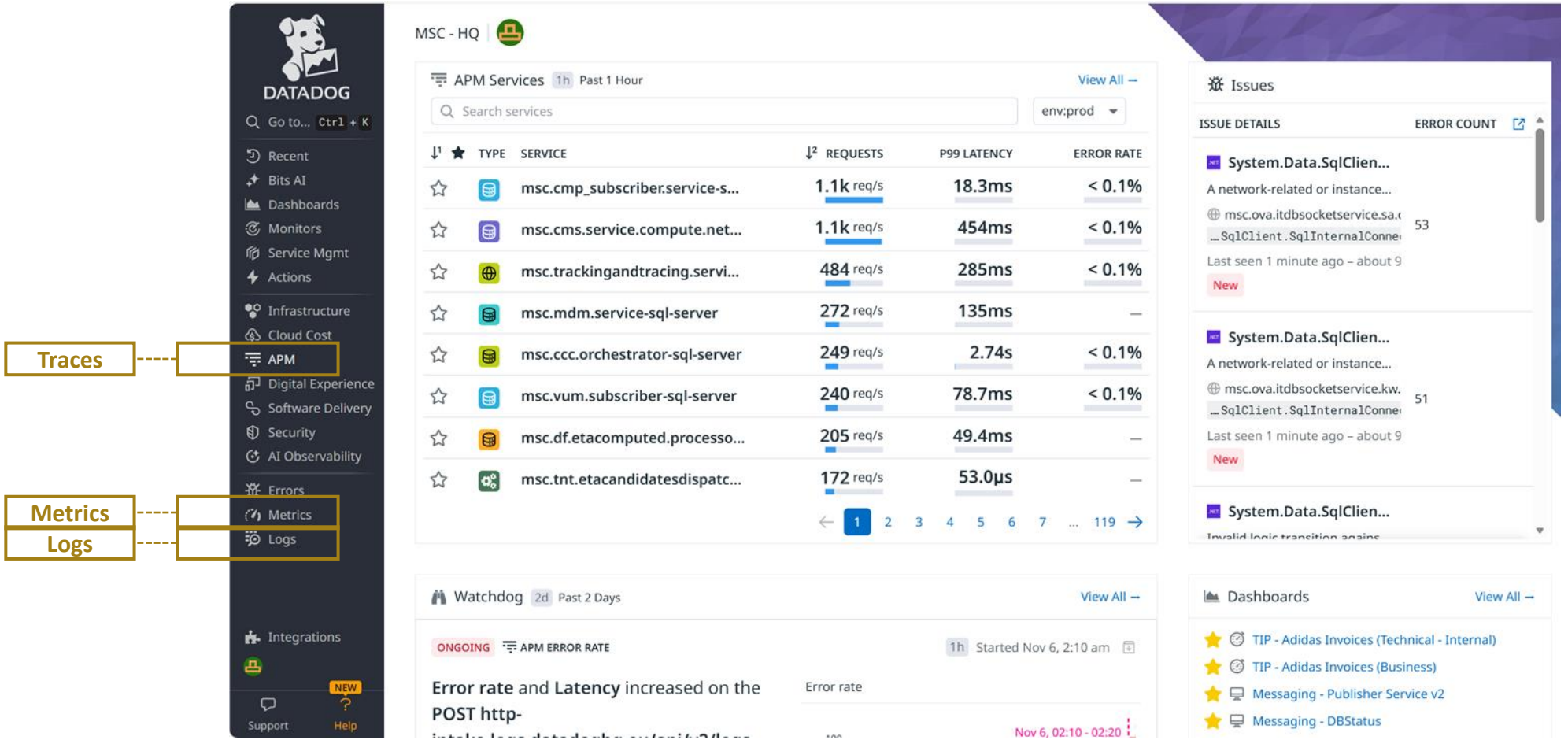
APPLICATION INSIGHTS - HOMEPAGE



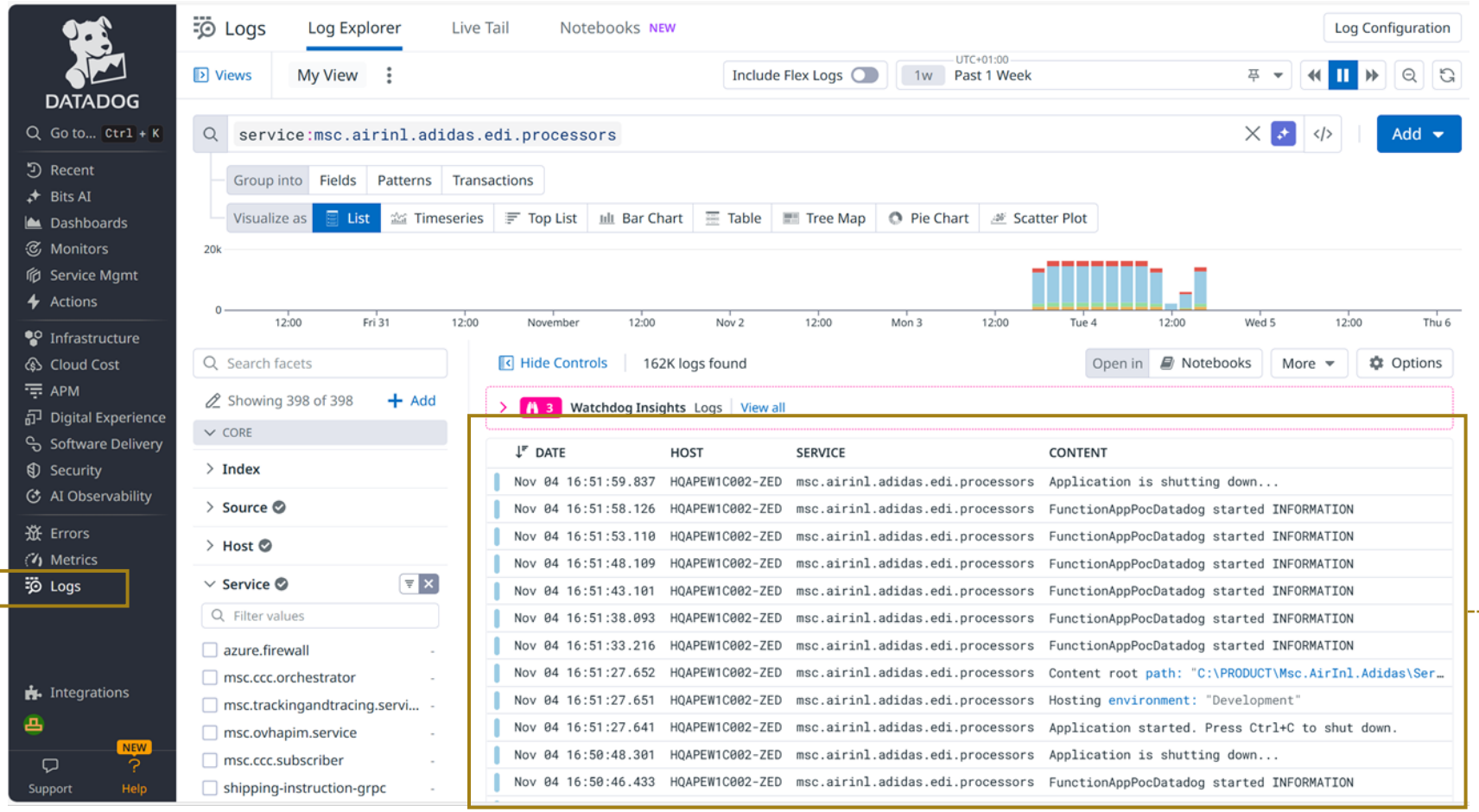
APPLICATION INSIGHTS - HOMEPAGE



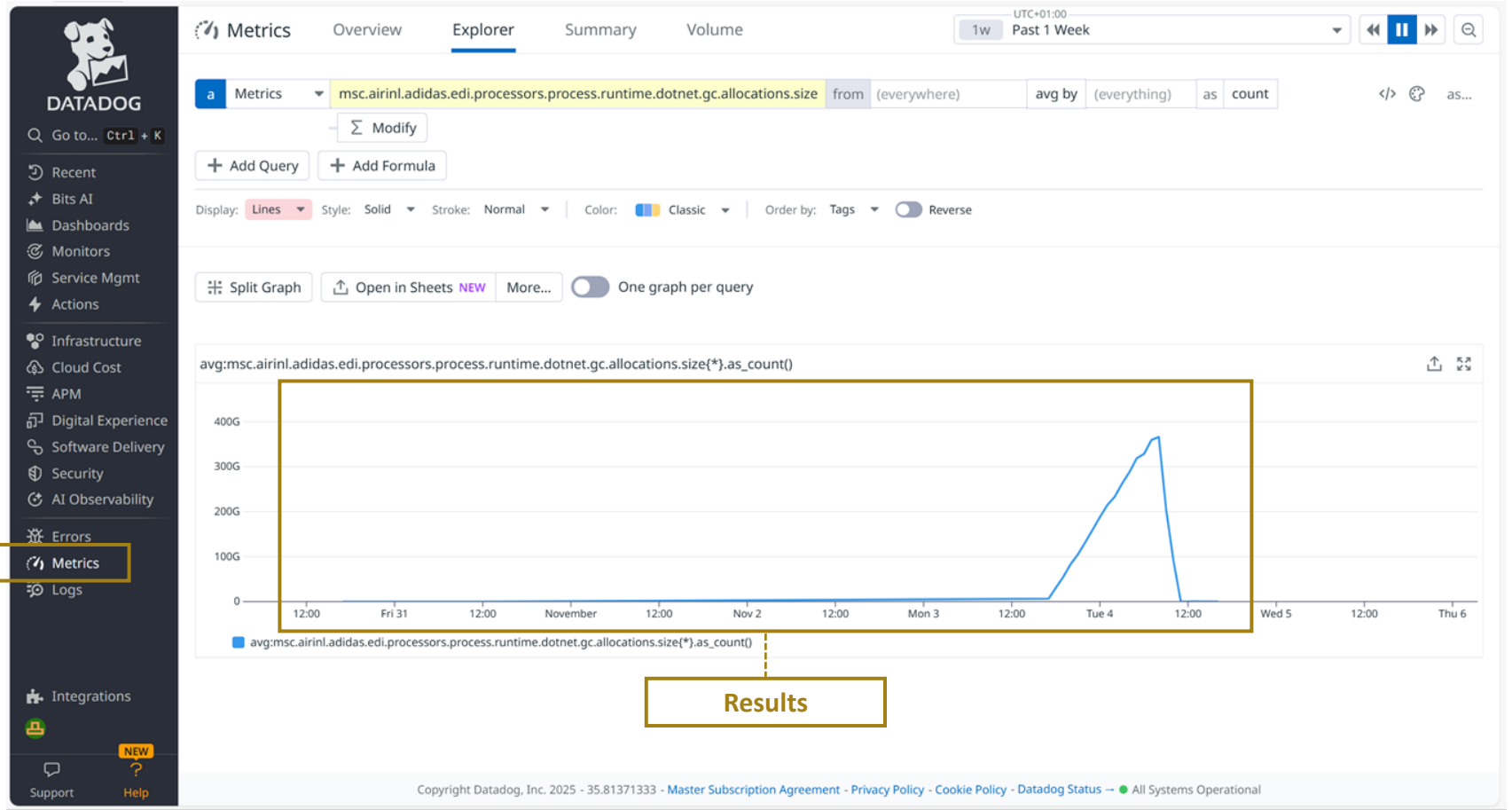
DATADOG - HOMEPAGE



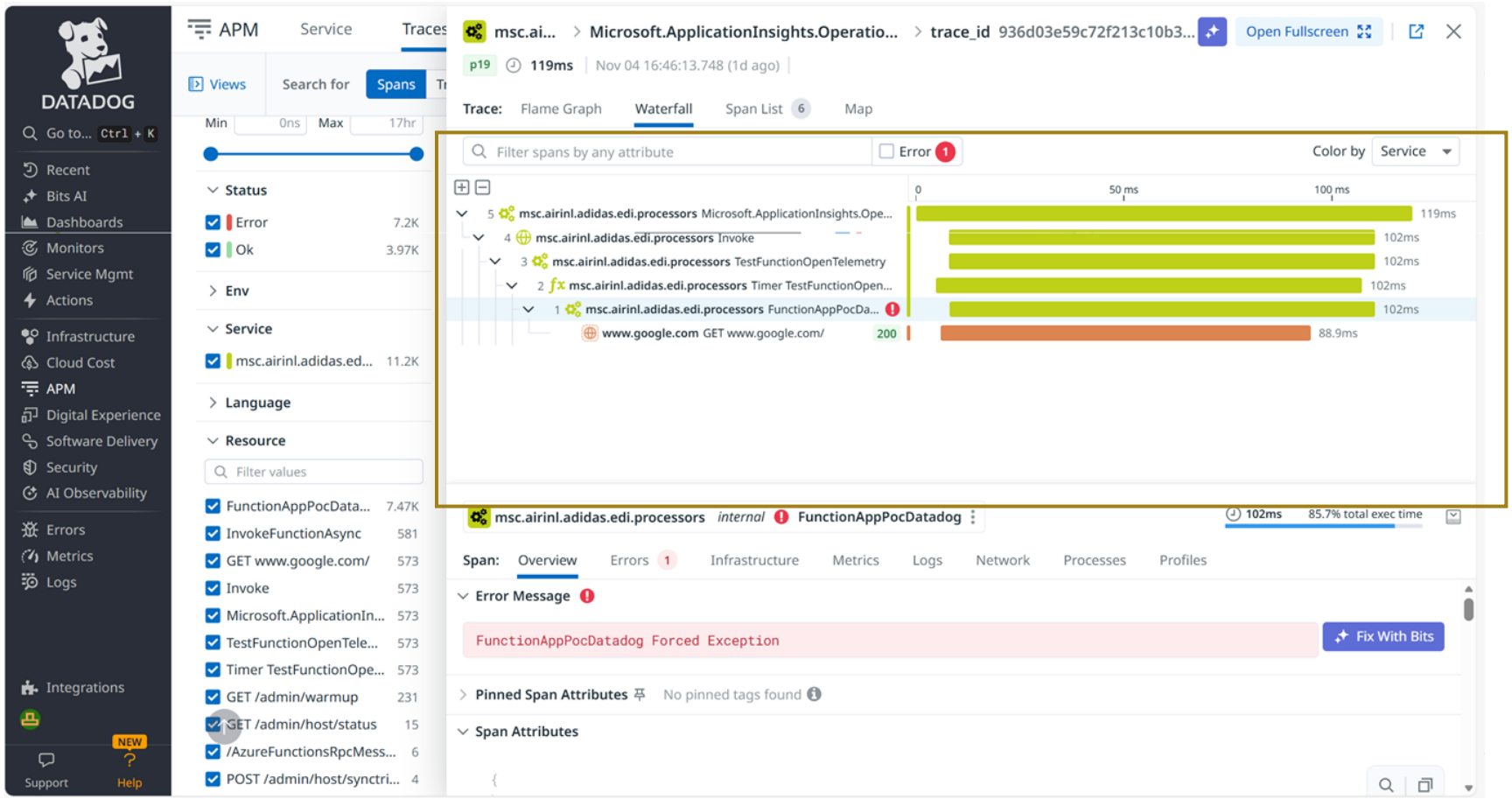
DATADOG - LOGS



DATADOG - METRICS



DATADOG - TRACES



DATADOG – GETTING STARTED

In Datadog, there are **two sides of the same coin** that every developer should understand:

1. **Sending data to DataDog**
   - Be able to send from your code **logs, metrics, and traces** to Datadog.

2. **Using DataDog**
   - **Use Datadog to observe** how your code behaves, identify issues, and monitor performance.

DATADOG – USING DATA

- **Concept of "Tag":**
  - Tags in Datadog are **key-value pairs attached to** logs, metrics and traces.
  - It's a way to **group together logs, metrics and traces**.

- **Unified Service Tagging**
  - Unified Tagging means using (at least) the **three mandatory tags** for all the logs, metrics and traces:
    - **"env"**
      - E.g.: "env: ci"
      - E.g.: "env: production"
    - **"service"**
      - Unique name attached to the application (or modules of my application)
      - E.g.: "service: myapplication.frontend"
      - E.g.: "service: myapplication.backend"
    - **"version"**
      - The specific version of my application
      - E.g.: "version: 0.0.0"

A quick digression…about…

ENVIRONMENTS

ENVIRONMENTS

What:

- An environment is a **copy of the same application** deployed on different machines.
- Environments are separate instances of the same code, deployed independently for safe development, testing, and production.
  - "**Dev** (Development)": Environment where developers write and test new features locally.
  - "**CI** (Continuous Integration)": Automated build and unit test environment to validate code after each commit.
  - "**TST** (Test)": Environment for testing to catch bugs before user testing.
  - "**UAT** (User Acceptance Testing)": Environment where end users validate features before production.
  - "**PRE** (Pre-Production)": Mirror of production for final checks and performance testing.
  - "**PRD** (Production)": Live environment where the application is available to end users.

When:

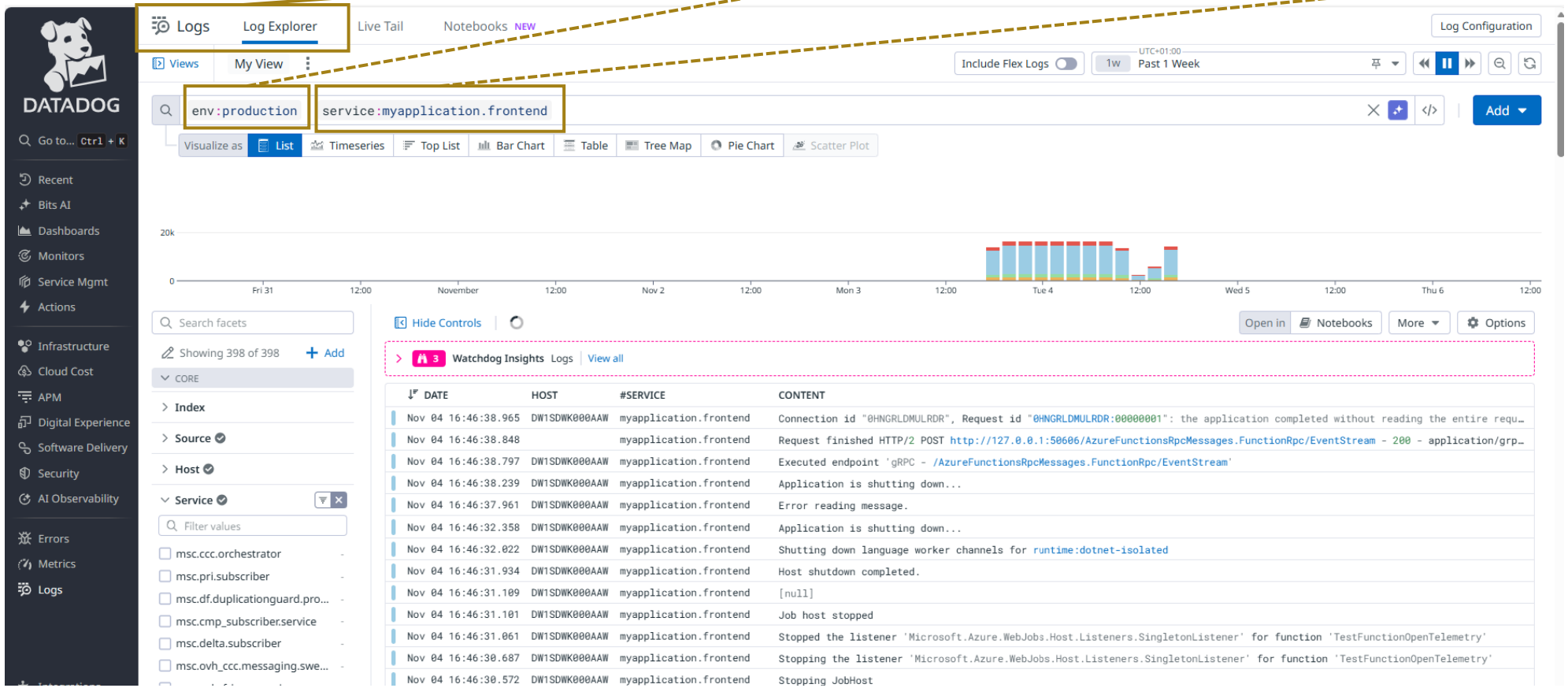
- Always :)

Why:

- To let each stage of the software life cycle (development, testing, and production) to run independently without affecting the others.

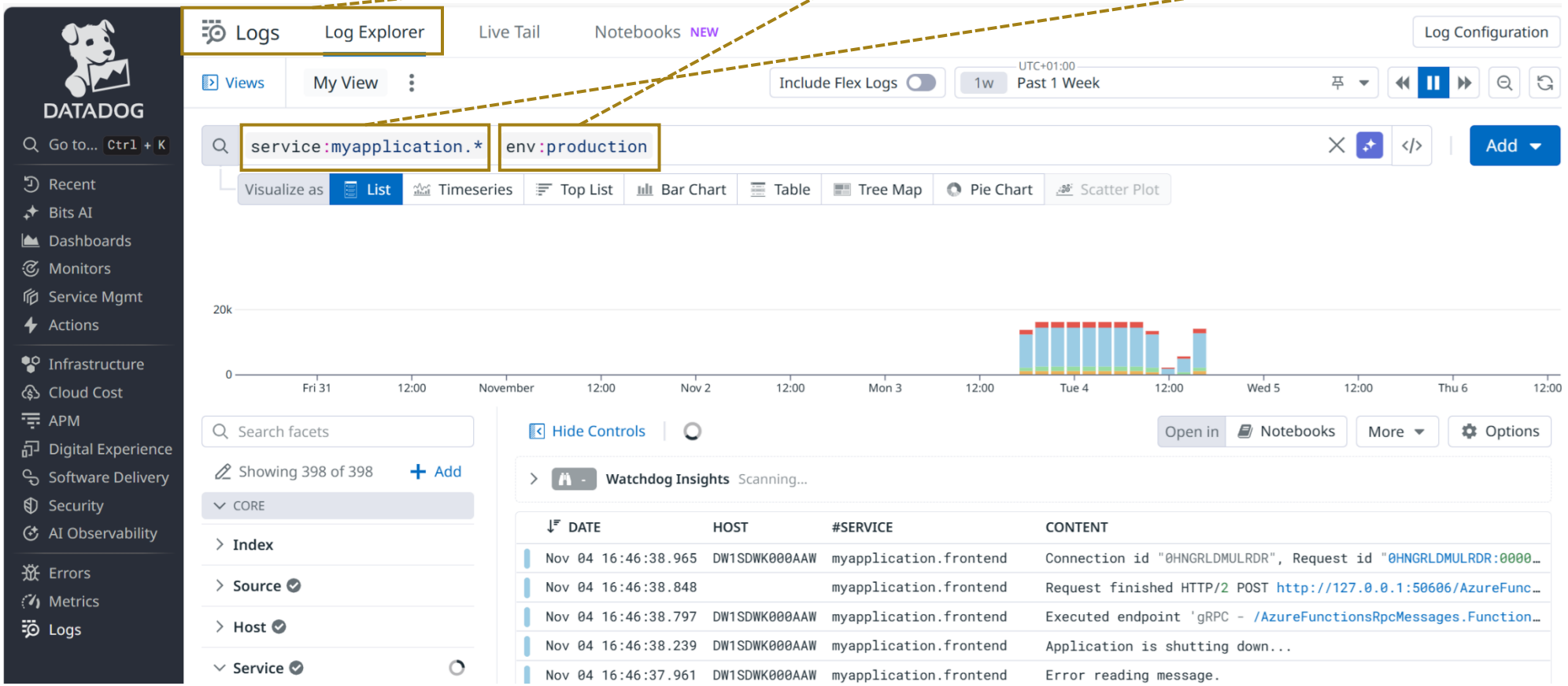
DATADOG – USING DATA – EXAMPLE 1

- **Example 1**: You need to troubleshoot an abnormal behavior in production, and the issue is coming from the frontend module.



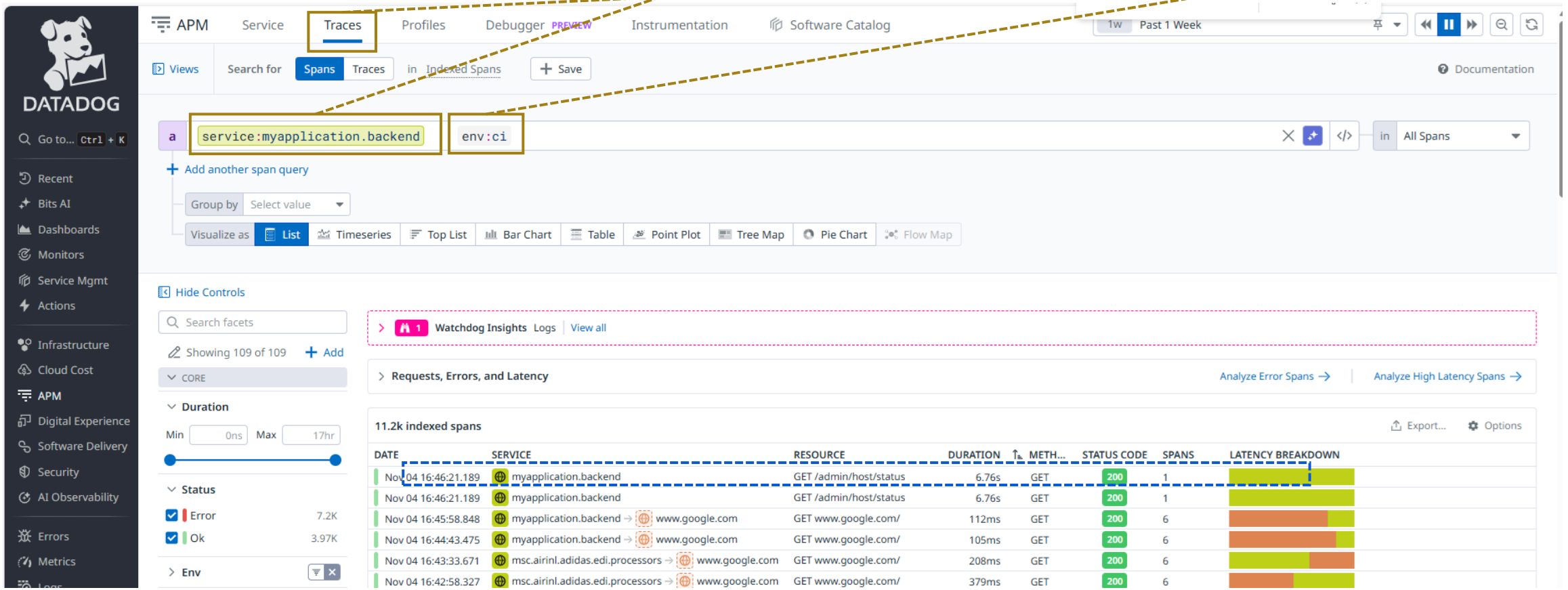
DATADOG – USING DATA – EXAMPLE 2

- **Example 2**: You need to troubleshoot an abnormal behavior in production, and you don't know where the issue is coming from



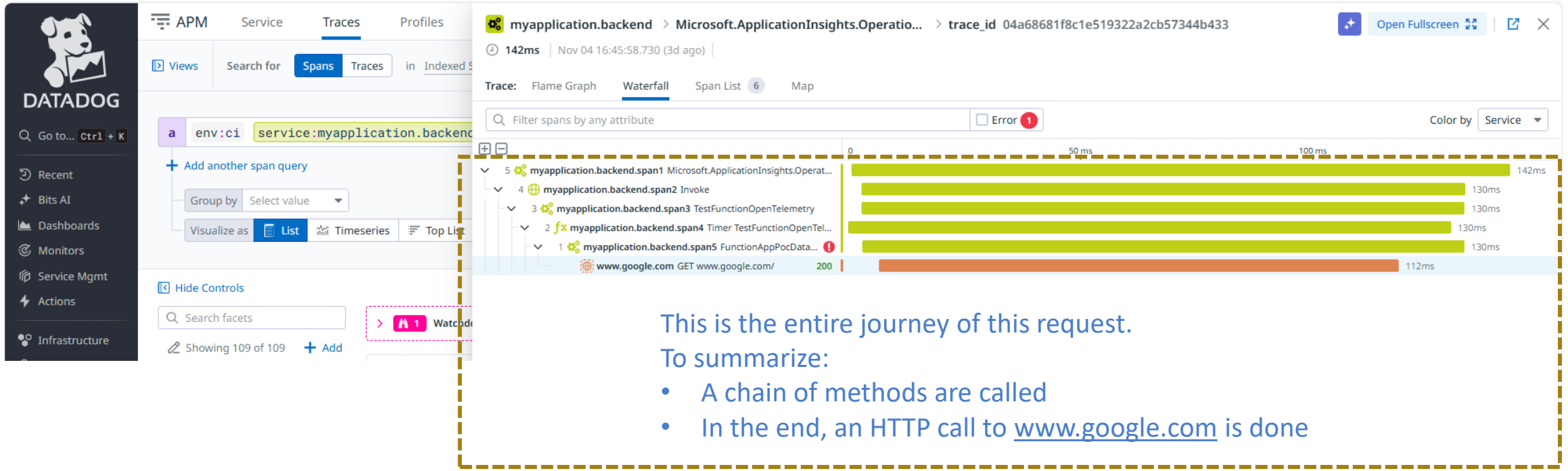
DATADOG – USING DATA – EXAMPLE 3

- **Example 3**: You need to troubleshoot why in the backend module an operation is takes too long, in CI environment



DATADOG – USING DATA – EXAMPLE 3

- **Example 3**: You need to troubleshoot why in the backend module an operation is takes too long, in CI environment

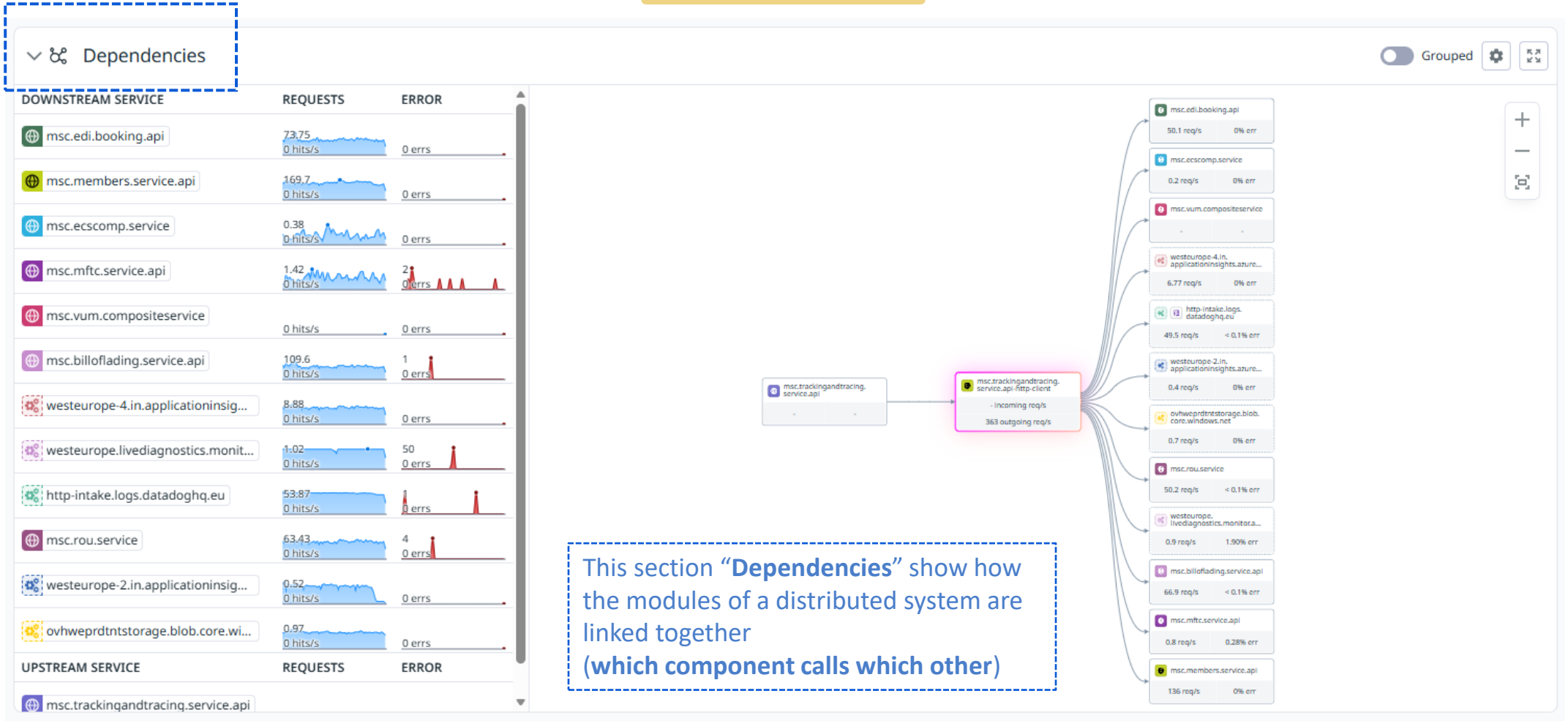


DATADOG

Monitoring tools are

not only logs, metrics, traces

DATADOG - DEPENDENCIES



This section "**Dependencies**" show how the modules of a distributed system are linked together
(**which component calls which other**)

LOG - IMPLEMENTATION

Let’s start with the implementation of **logging in C#**.

LOG - IMPLEMENTATION

First some concepts:

- **ILogger**:
  - It is the **main interface in .NET for writing logs**

- **Logging Providers**:
  - A provider defines **where** the logs go
    - ConsoleLoggerProvider → Logs go to the Console
    - FileLoggerProvider → Logs go to a file

- **Factory**:
  - A **Factory is an object that creates other objects**.
    - The Factory hides the object creation details, so you don't repeat them every time you create an object.
  - **LoggerFactory**
    - Object that creates loggers
    - With LoggerFactory, I create a logger and attach multiple providers to it.

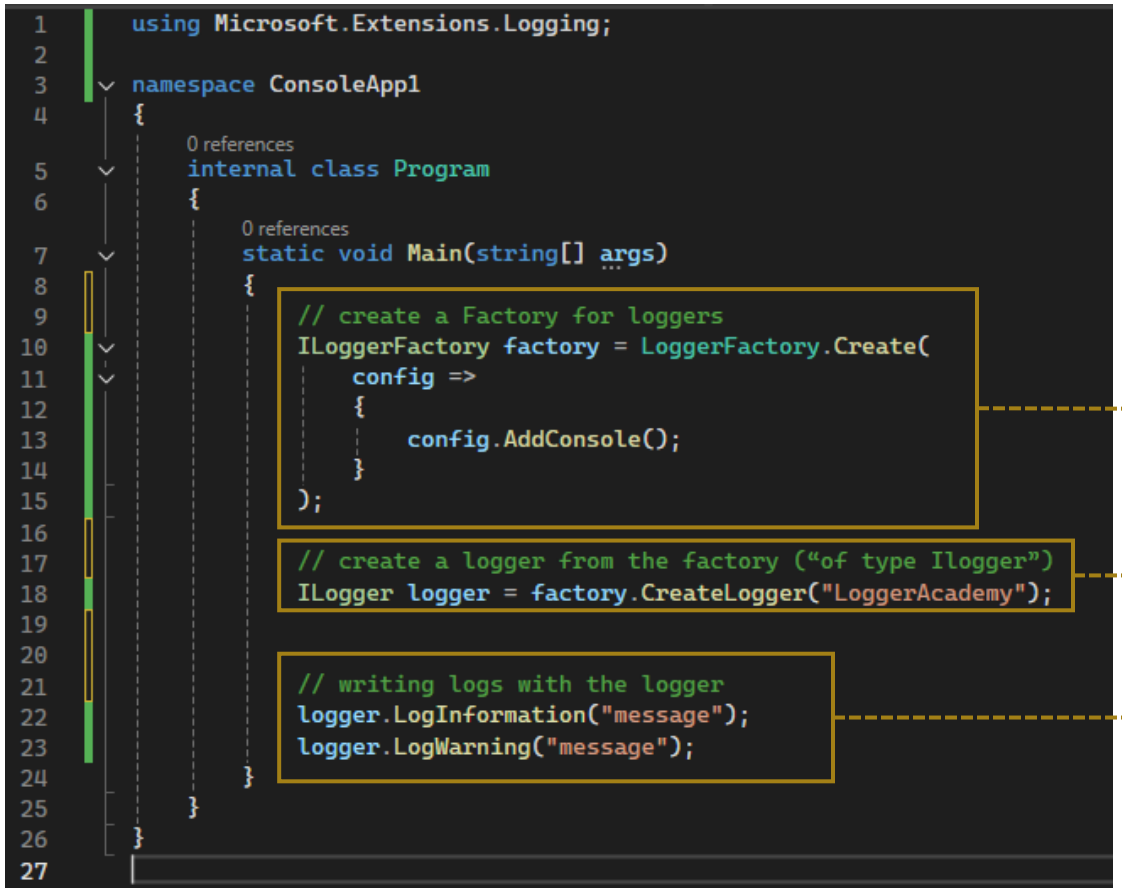
LOG - IMPLEMENTATION

Nuget Package to install:

- Microsoft.Extensions.Logging
- Microsoft.Extensions.Logging.Console
- And all other packages required for logging to different destinations (see details below).

LOG - IMPLEMENTATION

In a Console App you must manually create a logger:



```
using Microsoft.Extensions.Logging;

namespace ConsoleApp1
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            // create a Factory for loggers
            ILoggerFactory factory = LoggerFactory.Create(
                config =>
                {
                    config.AddConsole();
                }
            );

            // create a logger from the factory ("of type Ilogger")
            ILogger logger = factory.CreateLogger("LoggerAcademy");


            // writing logs with the logger
            logger.LogInformation("message");
            logger.LogWarning("message");
        }
    }
}
```

- These are the "details" hidden in the Factory
- In this case adding two Logging Providers (where logs are sent)
- So, every time I create a logger with the Factory, I receive a logger that send to two providers

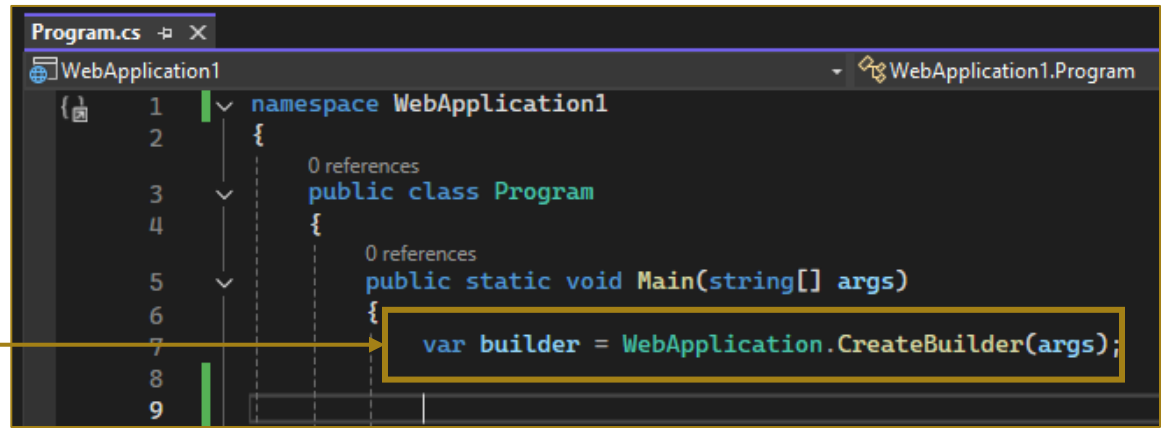
- "Hey Factory, please create a Logger"
- "LoggerAcademy" is a string that's associated with each message logged by the ILogger

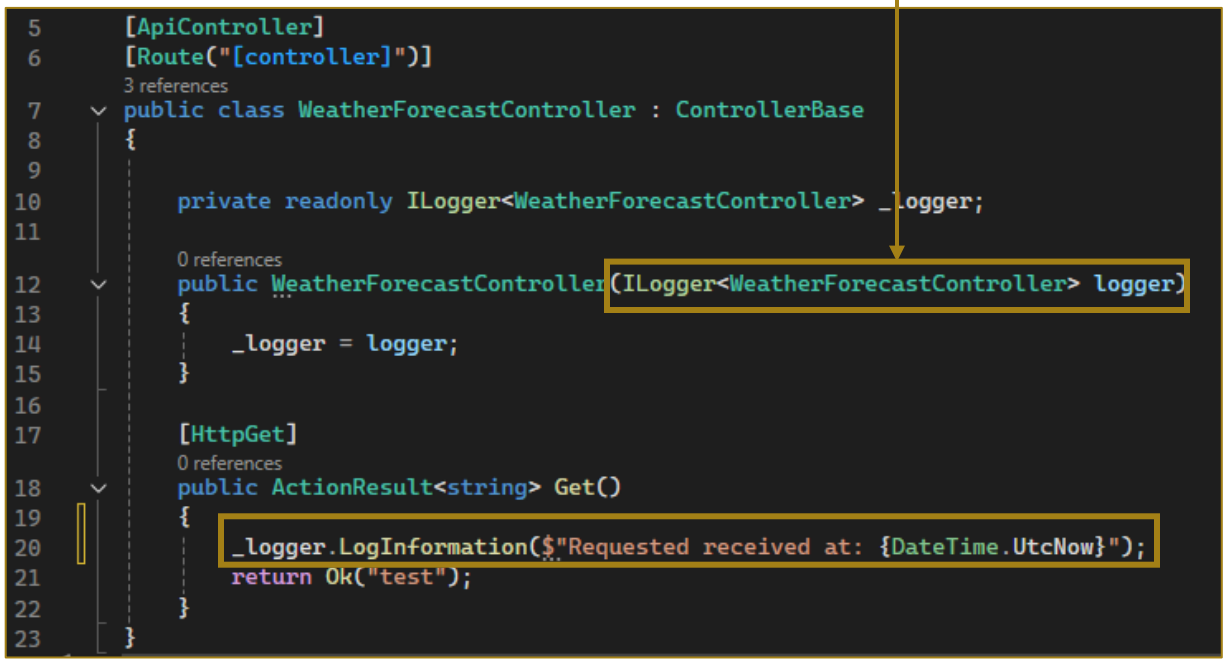
- ILogger has some methods to log using the different Log Level
- logger.LogInformation("") → Information Log Level
- logger.LogError("") → Error Log Level

LOG - IMPLEMENTATION

In **ASP.NET controllers**:

- You **don't need to create manually a logger**
- The Host in Program.cs (via CreateDefaultBuilder()) automatically:
  - Configures some default logging providers (like Console)
  - Injects the ILogger interface into your classes (like controllers or services)





LOG - CONFIGURATION

Configure the logging:

- What does it mean?
  - **Minimum Log Level** → **Only the logs with a higher log level will be displayed**
  - **Configure a provider** → E.g.: the **Console provider should have a timestamp on each log**
- If you need to **configure logging**, how can you do it?
  1. Via configuration file appsettings.json → The Host automatically read the configuration
  2. Via code

appsettings.json*

Schema: https://www.schemastore.org/appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Error"
    },
    "Console": {
      "TimestampFormat": "[HH:mm] "
    }
  }
}
```

```
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Set the default log level for all providers
        builder.Logging.SetMinimumLevel(LogLevel.Error);

        // Customize the existing Console provider
        builder.Logging.AddSimpleConsole(options =>
        {
            options.TimestampFormat = "[HH:mm] ";
        });
```

LOG - CONFIGURATION

Configure the logging:

- What does it mean?
  - **Minimum Log Level** → Only the logs with a higher log level will be displayed
  - **Configure a provider** → E.g.: the Console provider should have a timestamp on each log
- If you need to **configure logging**, how can you do it?
  1. Via configuration file appsettings.json → The Host automatically read the configuration
  2. Via code

Log not showed since it has a < LogLevel than error

At the beginning of each log row there is a timestamp "[HH:mm]"

```
[HttpGet]
0 references
public ActionResult<string> Get()
{
    _logger.LogInformation($"Requested INFO received at: {DateTime.UtcNow}");

    _logger.LogError($"Requested ERROR received at: {DateTime.UtcNow}");

    return Ok("test");
}
```

```
[14:36] fail: WebApplication1.Controllers.WeatherForecastController[0]
      Requested ERROR received at: 11/7/2025 1:36:35 PM
```

Log showed since it has a >= LogLevel than error

LOG – STRUCTURED LOG

Structured logging with ILogger:

- First, it's need to add Logging Provider that can "accept" Structure Logging. One of them is "JsonConsole".
- You must write logs with message template and arguments (NOT string interpolation).

```
public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);

    // Customize the existing Console provider
    builder.Logging.AddJsonConsole(options =>
    {
        options.TimestampFormat = "[HH:mm] ";
    });
```

```
[HttpGet]
0 references
public ActionResult<string> Get()
{
    _logger.LogInformation("Requested INFO received at: {_placeholder_now_}", DateTime.UtcNow);

    return Ok("test");
}
```

"_placeholder_now" is not just a value that is replaced, **but it's an attribute you can search on**.

LOG - SERILOG

The standard .NET ILogger is limited in complex scenario

…

You can use third party solutions

LOG - SERILOG

Serilog:

- Why use a third-party solution like Serilog?
  - Rich of sinks (**destination of your logs**)
    - Console
    - Database (like SQL)
    - Datadog
    - …
  - Powerful configuration
  - Better for Production
    - Async writes
    - Designed for huge volume of data

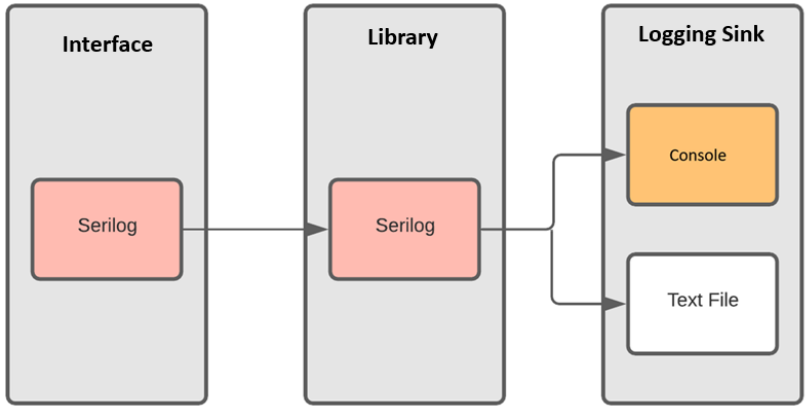
- **Serilog.Sinks.ApplicationInsights**
- **Serilog.Sinks.Datadog.Logs**
- **Serilog.Sinks.Console**
- Serilog.Sinks.AzureTableStorage
- Serilog.Sinks.Elasticsearch
- Serilog.Sinks.ElmahIo
- Serilog.Sinks.Email
- Serilog.Sinks.File
- Serilog.Sinks.GoogleCloudLogging
- Serilog.Sinks.InMemory
- Serilog.Sinks.Log4Net
- Serilog.Sinks.Loggly
- Serilog.Sinks.MongoDB
- Serilog.Sinks.NLog
- Serilog.Sinks.PostgreSQL
- Serilog.Sinks.RabbitMQ
- Serilog.Sinks.RollingFile
- Serilog.Sinks.Seq
- Serilog.Sinks.SignalR
- Serilog.Sinks.SQLite
- Serilog.Sinks.Stackify
- …

LOG - SERILOG

How to use Serilog (two options):

1) Directly

- Your code (classes, services, controllers, …) will contain specific Serilog code
- If you decide to change your logging library (Serilog → Log4Net) you must change all the points of your code where Serilog is present.



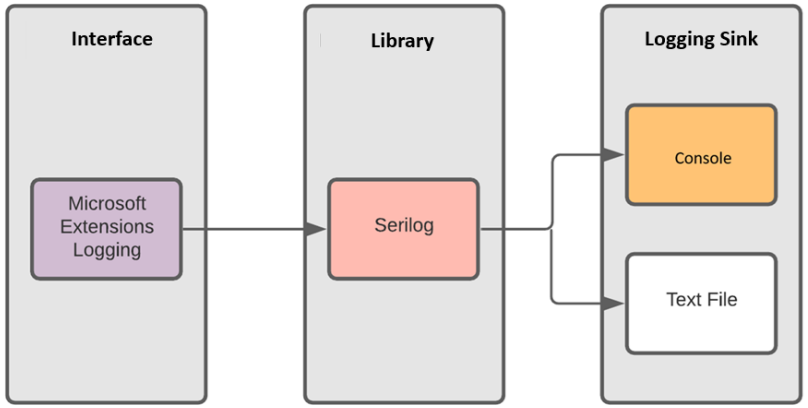
Serilog vs. Microsoft Extensions Logging: Which to Use? - Loupe

2) Integrated with ILogger [Recommended]

- **Use Serilog as a provider of ILogger**
  - Previously we saw how ILogger sends logs to the Console provider that shows the logs in the terminal
  - Serilog will be like another provider, so ILogger send logs to Serilog that send to the right sinks

- Your code (classes, services, controllers, …) will contain ILogger methods

- You still have to "link" Serilog and ILogger in the Program.cs



Serilog vs. Microsoft Extensions Logging: Which to Use? - Loupe

LOG - SERILOG

How to use Serilog (option 2):

Nuget packages to install:

- Serilog.AspNetCore (To use Serilog in ASP.NET project)

In the **Program.cs** you must:

1. Configure Serilog (such as the sinks…)
2. Set Serilog as a provider of ILogger

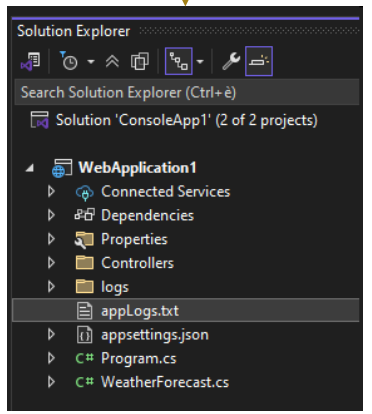
```
public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);

    var configSerilog = new LoggerConfiguration()
        .WriteTo.Console()
        .WriteTo.File("appLogs.txt")
        .CreateLogger();

    builder.Services.AddSerilog(configSerilog);
```

We're saying:

- Add the Console Sink
- Add the File Sink (the file name will be "appLogs.txt")

```
[HttpGet]
0 references
public ActionResult<string> Get()
{
    _logger.LogInformation("Requested INFO received at: {_placeholder_now_}", DateTime.UtcNow);

    return Ok("test");
}
```

- We didn't change the code where we log, since we continued to use ILogger.
- But now, ILogger "sends" the log to "Serilog" that sends the log to the file sink, so writing to a file



LOG - SERILOG

How to use Serilog (option 2) with DataDog:

Nuget packages to install:

- Serilog.AspNetCore (To use Serilog in ASP.NET project)
- Serilog.Sinks.Datadog.Logs (to use Datadog as sink)

In the **Program.cs** you must:

1. Configure Serilog (such as the sinks…)
2. Set Serilog ad a provider of ILogger

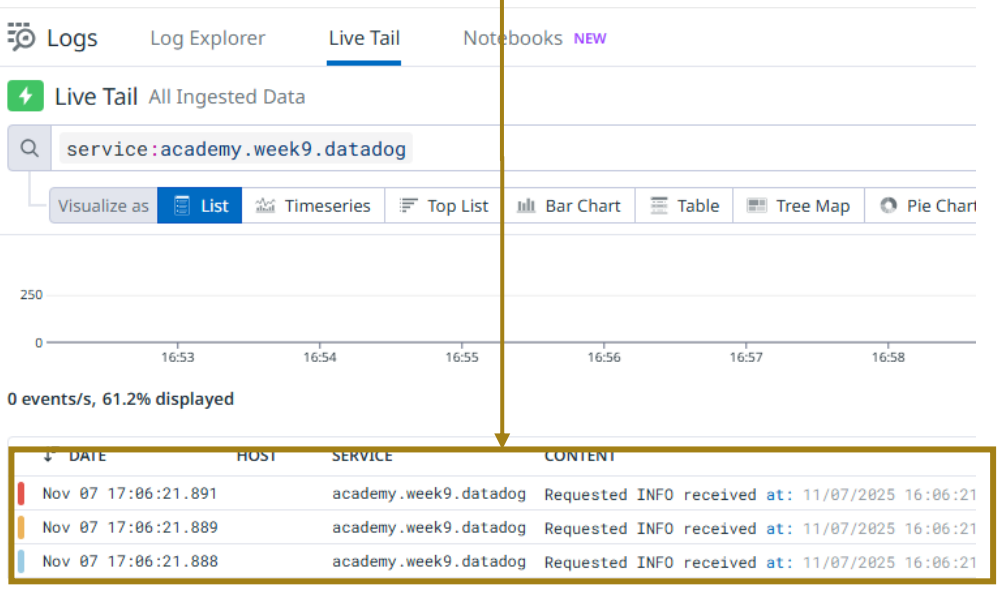
Don't try to memorize it, let's focus on understanding the concept first!

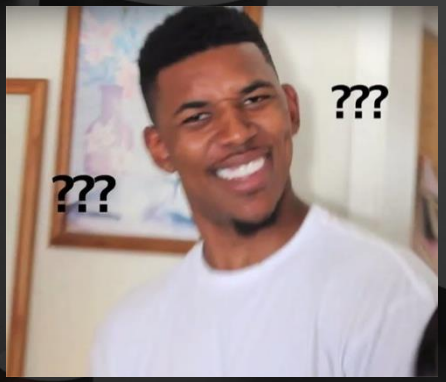
We're saying to Serilog:

- Use Datadog as sink of your logs
- But we have also to "configure" how Serilog connects to Datadog
  - apiKey: the "password" to be able to push on Datadog
  - service: do you remember the UMF of Datadog? It's a mandatory field
  - URL and Port: where Datadog can be contacted

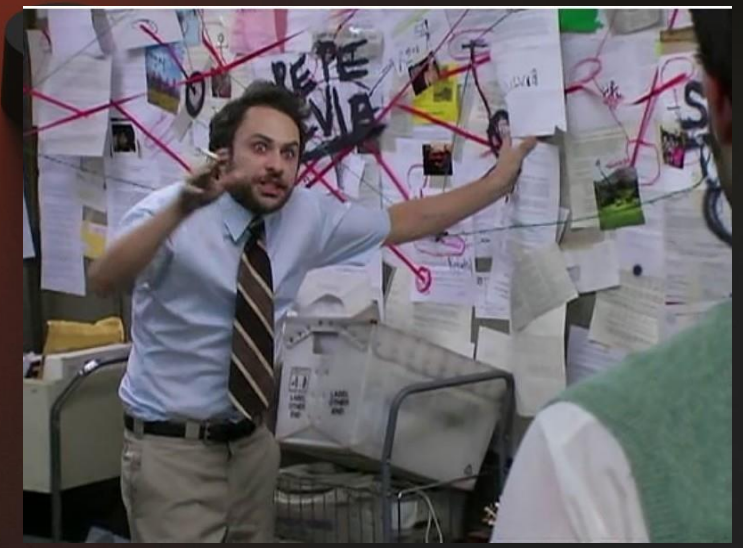
```
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // create configuration for Serilog
        var configSerilog = new LoggerConfiguration()
            .WriteTo.Console()
            .WriteTo.DatadogLogs
            (
                apiKey: "[redacted]",
                service: "academy.week9.datadog",
                configuration: new DatadogConfiguration()
                {
                    Url = "https://http-intake.logs.datadoghq.eu",
                    Port = 10516
                }
            )
            .CreateLogger();

        builder.Services.AddSerilog(configSerilog);
```

```
[HttpGet]
0 references
public ActionResult<string> Get()
{
    _logger.LogInformation("Requested INFO received at: {_placeholder_now_}", DateTime.UtcNow)
    _logger.LogWarning("Requested INFO received at: {_placeholder_now_}", DateTime.UtcNow);
    _logger.LogError("Requested INFO received at: {_placeholder_now_}", DateTime.UtcNow);

    return Ok("test");
}
```





Q&A
Time



LOGS – EXERCISE

All the exercises have the same structure:

1. **Program.cs**
    A. Configure the Logging

2. **WeatherForecastController.cs**
    A. Controller with the HTTP endpoint "*GetWeatherForecastPlusDays*" exposed
    B. The endpoint receives the request and calls the "*WeatherForecastService*" service

3. **WeatherForecastService.cs**
    A. The "*WeatherForecastService*" service returns **as many items as the number of days between the current date and *plusDays.***

LOGS – EXERCISE - 1.1.LOGGING.MICROSOFT.SDK

Exercise “1.1.Logging.Microsoft.SDK ” – Focal Points

- **Nuget**:
  - No additional Nuget packages needed

- **Program.cs**
  - Setup ILogger

```
0 references
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
```

- **WeatherForecastController.cs**
  - Logging the request received

```
[HttpGet(Name = "GetWeatherForecastPlusDays")]
0 references
public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, int day, int plusDays)
{
    _logger.LogInformation($"Requested received at {DateTime.UtcNow}");

    IList<WeatherForecast> results = _weatherForecastService.GetWeatherForecastPlusDays(year, month, day, p

    return results;
}
```

LOGS – EXERCISE - 1.2.LOGGING.MICROSOFT.SDK.STRUCTURED

Exercise “1.2.Logging.Microsoft.SDK.Structured” – Focal Points

- **Nuget**:
  - No additional Nuget packages needed

- **Program.cs**
  - Setup ILogger
  - Add “SimpleConsole” logging provider
  - Add “JsonConsole” logging provider

```
// add structured log console visualizer
builder
    .Logging
    // Remove all the Logging Providers before adding the ones
    // to be sure to use only the ones added by me and not auto
    .ClearProviders()
    .AddSimpleConsole() // <-- to see the "plain string" log
    .AddJsonConsole // <-- add another logging provider to see
    (
        c => c.JsonWriterOptions = new() { Indented = true } /
    );
```

- **WeatherForecastController.cs**
  - Logging the request received

- **Output**

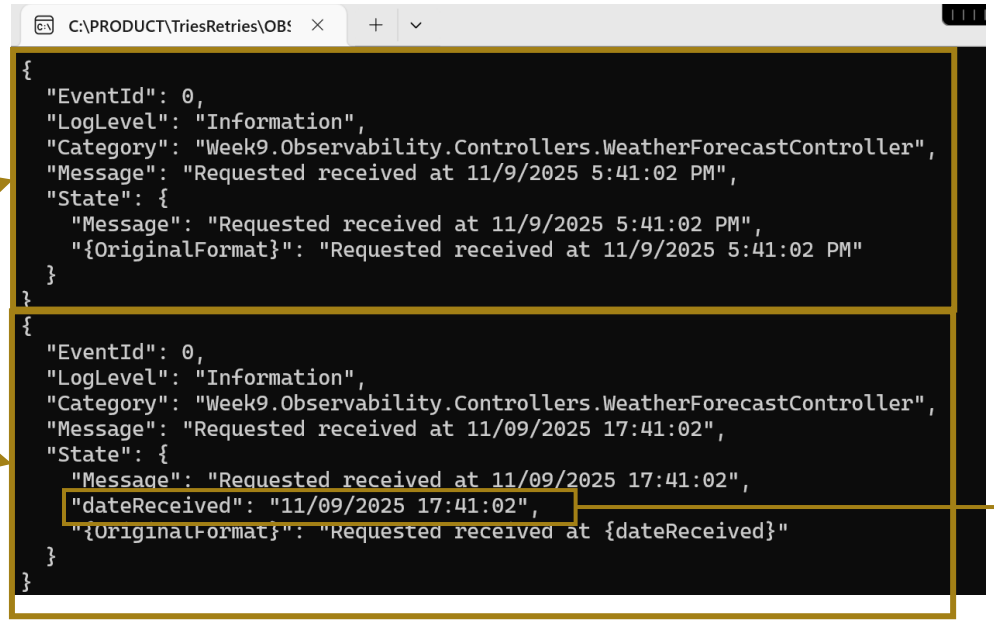
```
[HttpGet(Name = "GetWeatherForecastPlusDays")]
0 references
public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, int day, int plusDa
{
    // WRONG -> NOT use string interpolation otherwise it's now a STRUCTURED log but only a single stri
    _logger.LogInformation($"Requested received at {DateTime.UtcNow}");

    // CORRECT - but use template with params instead
    _logger.LogInformation("Requested received at {dateReceived}", DateTime.UtcNow);


    IList<WeatherForecast> results = _weatherForecastService.GetWeatherForecastPlusDays(year, month, da

    return results;
}
```

LOGS – EXERCISE - 1.2.LOGGING.MICROSOFT.SDK.STRUCTURED

Exercise “1.2.Logging.Microsoft.SDK.Structured” – Focal Points

- **Nuget**:
  - No additional Nuget packages needed
- **Program.cs**
  - Setup ILogger
  - Add “SimpleConsole” logging provider
  - Add “JsonConsole” logging provider
- **WeatherForecastController.cs**
  - Logging the request received
- **Output**

- “**dateReceived**” it’s an attribute you can search on.
- It’s present **only in** the **structured** log



LOGS – EXERCISE - 1.3.LOGGING.SERILOG

Exercise “1.3.Logging.Serilog” – Focal Points

- **Nuget**:
  - Serilog
  - Serilog.Extensions.Hosting (to use .AddSerilog())
  - Serilog.Sinks.Console (to use File sink)

- **Program.cs**
  - Setup **Serilog as** ILogger **logging provider**
  - Adding **Console** sink

```
// add structured log console visualizer
builder
   .Logging
   // Remove all the Logging Providers before adding the ones that i want
   // to be sure to use only the ones added by me and not automatically added by ASP.NET
   .ClearProviders();

// create configuration for Serilog
var configSerilog = new LoggerConfiguration()
    .WriteTo.Console() // <- The SINK added to Serilog is only the Console (for now...)
    .CreateLogger();

builder.Services.AddSerilog(configSerilog); // <-- This method, provided by Serilog Nuget,
```

- **WeatherForecastController.cs**
  - Logging the request received

```
[HttpGet(Name = "GetWeatherForecastPlusDays")]
0 references
public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, in
{
    // CORRECT - but use template with params instead
    _logger.LogInformation("Requested received at {dateReceived}", DateTime.UtcNow);

    IList<WeatherForecast> results = _weatherForecastService.GetWeatherForecastPlusDay

    return results;
}
```

LOGS – EXERCISE - 1.4.LOGGING.SERILOG.DATADOG

Exercise “1.4.Logging.Serilog.Datadog” – Focal Points

- **Nuget**:
  - Serilog
  - Serilog.Extensions.Hosting (to use .AddSerilog())
  - Serilog.Sinks.Console (to use File sink)
  - Serilog.Sinks.Datadog.Logs (to use DataDog sink)

- **Program.cs**
  - Setup **Serilog as** ILogger **logging provider**
  - Adding **Console** sink
  - Adding **DataDog** sink

- **WeatherForecastController.cs**
  - Logging the request received

- **Output**

```
// add structured log console visualizer
builder
    .Logging
    // Remove all the Logging Providers before adding the ones that i want
    // to be sure to use only the ones added by me and not automatically added by ASP.NET
    .ClearProviders();

// create configuration for Serilog
var configSerilog = new LoggerConfiguration()
    .WriteTo.Console() // <- SINK CONSOLE added to Serilog
    .WriteTo.DatadogLogs  // <- SINK DATADOG added to Serilog (so Serilog will push both o
    (
        apiKey: [redacted],
        service: "academy.week9.datadog",
        configuration: new DatadogConfiguration()
        {
            Url = "https://http-intake.logs.datadoghq.eu",
            Port = 10516
        }
    )
    .CreateLogger();

builder.Services.AddSerilog(configSerilog); // <-- This method, provided by Serilog Nuget,
```

```
[HttpGet(Name = "GetWeatherForecastPlusDays")]
0 references
public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, in
{
    // CORRECT - but use template with params instead
    _logger.LogInformation("Requested received at {dateReceived}", DateTime.UtcNow);

    IList<WeatherForecast> results = _weatherForecastService.GetWeatherForecastPlusDay

    return results;
}
```

LOGS – EXERCISE - 1.4.LOGGING.SERILOG.DATADOG

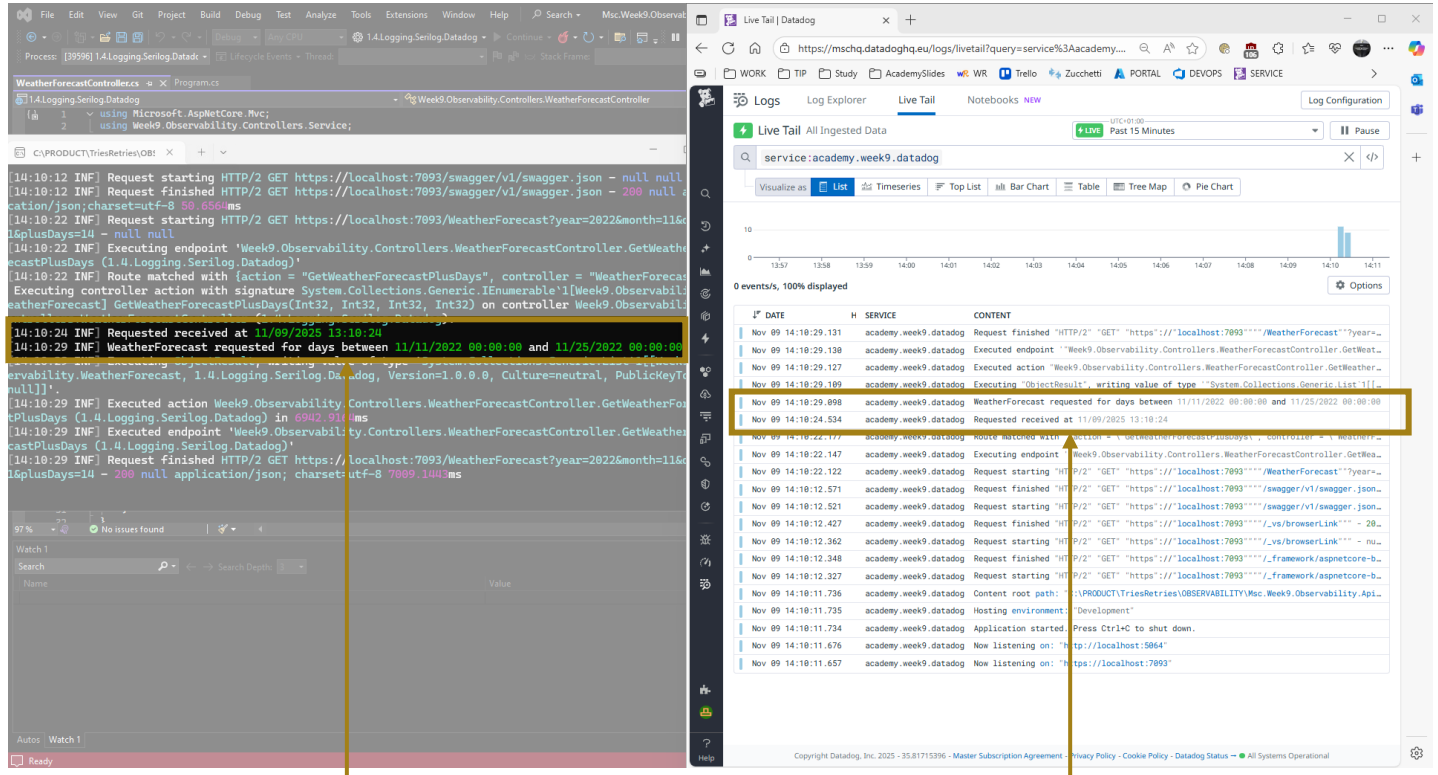
Exercise “1.4.Logging.Serilog.Datadog” – Focal Points

- **Nuget**:
  - Serilog
  - Serilog.Extensions.Hosting (to use .AddSerilog())
  - Serilog.Sinks.Console (to use File sink)
  - Serilog.Sinks.Datadog.Logs (to use DataDog sink)

- **Program.cs**
  - Setup **Serilog as** ILogger **logging provider**
  - Adding **Console** sink
  - Adding **DataDog** sink

- **WeatherForecastController.cs**
  - Logging the request received

- **Output**



METRIC - IMPLEMENTATION

Let’s start with the implementation of **metrics in C#**.

METRIC – EXERCISE

All the exercises have the same structure:

1. **Program.cs**
   A. Configure the Metrics (and the logs using Serilog)

2. **WeatherForecastController.cs**
   A. Emit the metric "**weatherforecast.requested.total**" → To track every request arrived at the controller
   B. Emit the metric "**weatherforecast.requested.success**" → To track every request arrived at the controller
   C. Emit the metric "**weatherforecast.requested.error**" → To track every request arrived at the controller

METRICS – EXERCISE - 2.1.METRICS.MICROSOFT.SDK

Exercise "2.1.Metrics.Microsoft.SDK" – Focal Points

- **Nuget**:
  - OpenTelemetry.Exporter.Console
  - OpenTelemetry.Extensions.Hosting
- **Program.cs**
  - Setup OpenTelemetry
- **WeatherForecastController.cs**
  - In the constructor (DI) it creates 3 metrics
    - weatherforecast.requested.**total**
    - weatherforecast.requested.**success**
    - weatherforecast.requested.**error**
  - Every time a new request arrived in the controller:
    - weatherforecast.requested.**total** += 1
  - Every time a request is completed with Success
    - weatherforecast.requested.**success** += 1
  - Every time a request is completed with Error
    - weatherforecast.requested.**error** += 1

```
builder.Services
    .AddOpenTelemetry()
    .ConfigureResource(resource => resource.Clear())
    .WithMetrics(metrics =>
    {
        metrics
            .AddMeter("myapp.controller.metrics")
            .AddConsoleExporter(); // <-- required to see metrics in console
    });
```

Don't try to memorize it, let's focus on understanding the concept first!

- **OpenTelemetry**:
  - A library that helps your application collect and export observability data, like metrics (but also logs and traces)
- **AddOpenTelemetry()**:
  - This turns on OpenTelemetry in our app. It's like saying "start collecting metrics"
- **AddMeter()**:
  - Listen only this specific metric
- **AddConsoleExporter()**:
  - Export, every X seconds, the metrics collected in the Console

METRICS – EXERCISE - 2.1.METRICS.MICROSOFT.SDK

Exercise "2.1.Metrics.Microsoft.SDK" – Focal Points

- **Nuget**:
  - OpenTelemetry.Exporter.Console
  - OpenTelemetry.Extensions.Hosting
- **Program.cs**
  - Setup OpenTelemetry
- **WeatherForecastController.cs**
  - In the constructor (DI) it creates 3 metrics
    - weatherforecast.requested.**total**
    - weatherforecast.requested.**success**
    - weatherforecast.requested.**error**
  - Every time a new request arrived in the controller:
    - weatherforecast.requested.**total** += 1
  - Every time a request is completed with Success
    - weatherforecast.requested.**success** += 1
  - Every time a request is completed with Error
    - weatherforecast.requested.**error** += 1

```
[ApiController]
[Route("[controller]")]
5 references
public class WeatherForecastController : ControllerBase
{
    private readonly ILogger<WeatherForecastController> _logger;

    private readonly IWeatherForecastService _weatherForecastService;

    private readonly IMeterFactory _meterFactory;
    private readonly Counter<int> _counterRequestTotal;
    private readonly Counter<int> _counterRequestSuccess;
    private readonly Counter<int> _counterRequestError;

    0 references
    public WeatherForecastController(ILogger<WeatherForecastController> logger, IMeterFactory meterFactory, IWe
    {
        _logger = logger;
        _weatherForecastService = weatherForecastService;

        _meterFactory = meterFactory;
        var meter = meterFactory.Create("myapp.controller.metrics");
        _counterRequestTotal = meter.CreateCounter<int>("weatherforecast.requested.total");
        _counterRequestSuccess = meter.CreateCounter<int>("weatherforecast.requested.success");
        _counterRequestError = meter.CreateCounter<int>("weatherforecast.requested.error");
    }

    [HttpGet(Name = "GetWeatherForecastPlusDays")]
    0 references
    public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, int day, int plusDays)
    {
        Meter meter = _meterFactory.Create("myapp.controller.metrics");

        _logger.LogInformation("Requested received at {0}", DateTime.UtcNow);
        _counterRequestTotal.Add(1);
        IList<WeatherForecast> results = new List<WeatherForecast>();
        try
        {
            results = _weatherForecastService.GetWeatherForecastPlusDays(year, month, day, plusDays);
            _counterRequestSuccess.Add(1);
        }
        catch (Exception)
        {
            _counterRequestError.Add(1);
        }
        return results;
    }
```

① ② ③ ③ ③

Don't try to memorize it, let's focus on understanding the concept first!

1) Object **IMeterFactory** injected by DI. It's a factory, so it is used to create other objects…in this case **Meter**.

2) A **Meter is a logic group of metrics**. Every metrics (counters) generated by the Meter will be part of the same group. Starting from the Meter, **three different Metrics (Counters) are created**.

3) Metrics are increased with specific business logic. For example, every time a request is received the metric "total" is increased.

Let’s do a quick recap…

- In the “**2.1.Metrics.Microsoft.SDK**” exercise:
  - **Metrics are created** with tools already included in .NET (precisely “*System.Diagnostics.Metrics*”)
  - **Metrics are exported** with “*OpenTelemetry*” library, that hooks to “*System.Diagnostics.Metrics*” to collect and export those metrics wherever you want (Console)

- Now, we’ll do the same exercise but:
  - **Metrics are created** directly with **DataDog** library (the class that does the job is “**DogStatsdService**”)
  - **Metrics are exported** directly with **DataDog** library

METRICS – EXERCISE - 2.2.METRICS.DATADOG.SDK

Exercise "2.2.Metrics.Datadog.SDK" – Focal Points

- **Nuget**:
  - DogStatsD-CSharp-Client

- **Program.cs**
  - Setup DogStasD (the DataDog class that send metrics)

```
builder.Services.AddSingleton<IDogStatsd>(sp =>
{
    var statsdService = new DogStatsdService();
    var dogstatsdConfig = new StatsdConfig
    {
        StatsdServerName = "__TO_REPLACE__",
        StatsdPort = 8125,
        ServiceName = "academy.week9.datadog"
    };
    statsdService.Configure(dogstatsdConfig);

    return statsdService;
});
```

- **WeatherForecastController.cs**
  - In the constructor (DI) it creates 3 metrics
    - weatherforecast.requested.**total**
    - weatherforecast.requested.**success**
    - weatherforecast.requested.**error**
  - Every time a new request arrived in the controller:
    - weatherforecast.requested.**total** += 1
  - Every time a request is completed with Success
    - weatherforecast.requested.**success** += 1
  - Every time a request is completed with Error
    - weatherforecast.requested.**error** += 1

Don't try to memorize it, let's focus on understanding the concept first!

- **DogStatsdService**:
  - A class of the Datadog libraries that send the metrics

- **StatsdConfig**:
  - A class to setup the DogStatsD, like which DataDog server send the metrics

METRICS – EXERCISE - 2.2.METRICS.DATADOG.SDK

Exercise "2.2.Metrics.Datadog.SDK" – Focal Points

- **Nuget**:
  - DogStatsD-CSharp-Client

- **Program.cs**
  - Setup DogStasD (the DataDog class that send metrics)

- **WeatherForecastController.cs**
  - In the constructor (DI) it is passed the IDogStatsd
  - Every time a new request arrived in the controller:
    - weatherforecast.requested.**total** += 1
  - Every time a request is completed with Success
    - weatherforecast.requested.**success** += 1
  - Every time a request is completed with Error
    - weatherforecast.requested.**error** += 1

```
[ApiController]
[Route("[controller]")]
5 references
public class WeatherForecastController : ControllerBase
{
    private readonly ILogger<WeatherForecastController> _logger;

    private readonly IWeatherForecastService _weatherForecastService;
    private readonly IDogStatsd _dogStatsd;

    0 references
    public WeatherForecastController(ILogger<WeatherForecastController> logger, IDogStatsd dogStatsd, IWeatherF
    {
        _logger = logger;
        _weatherForecastService = weatherForecastService;
        _dogStatsd = dogStatsd;
    }

    [HttpGet(Name = "GetWeatherForecastPlusDays")]
    0 references
    public IEnumerable<WeatherForecast> GetWeatherForecastPlusDays(int year, int month, int day, int plusDays)
    {
        _logger.LogInformation("Requested received at {0}", DateTime.UtcNow);
        _dogStatsd.Counter("weatherforecast.requested.total", 1);
        IList<WeatherForecast> results = new List<WeatherForecast>();
        try
        {
            results = _weatherForecastService.GetWeatherForecastPlusDays(year, month, day, plusDays);
            _dogStatsd.Counter("weatherforecast.requested.success", 1);
        }
        catch (Exception)
        {
            _dogStatsd.Counter("weatherforecast.requested.error", 1);
        }
        return results;
    }
}
```

① ② ② ②

Don't try to memorize it, let's focus on understanding the concept first!

1) Object **IDogstasd** injected by DI. It's the interface of the DataDog library to use the service that send the metrics.

2) Every time it's needed, a metric is created and increased.

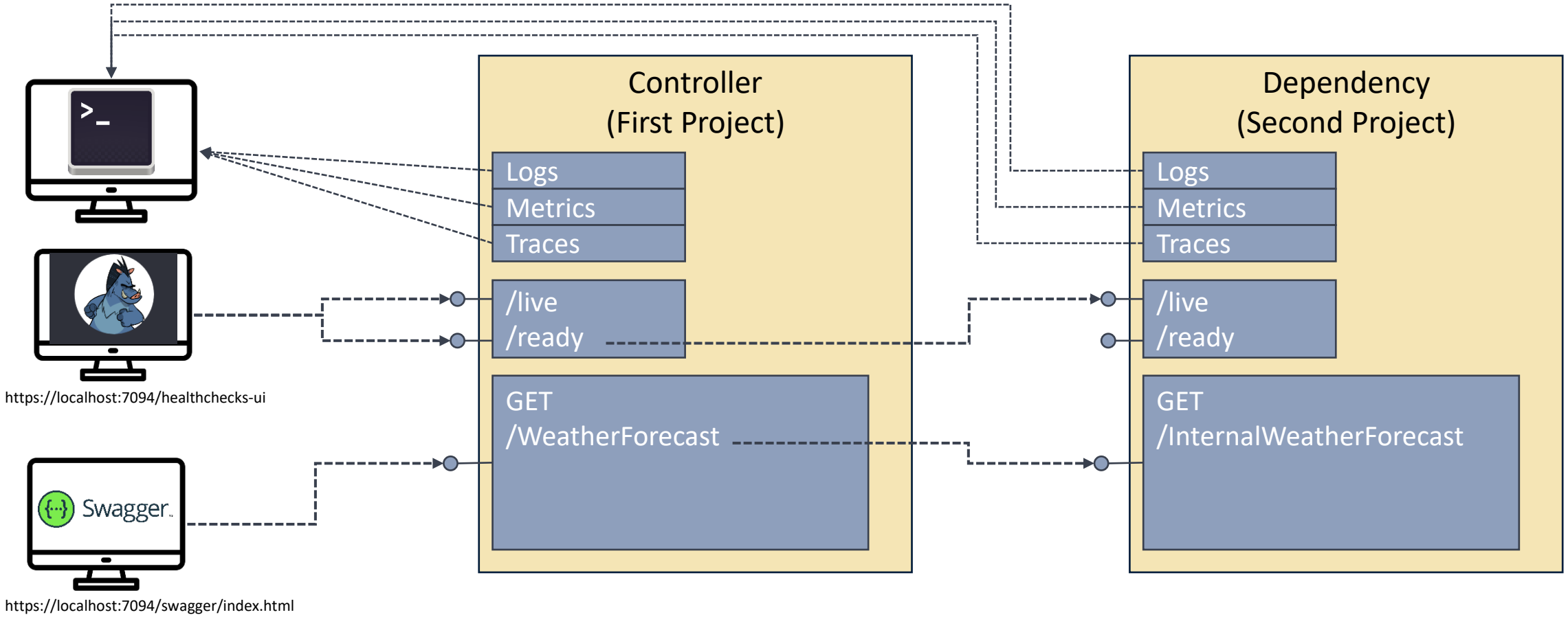
Naturally, when you create a metric with the same name, Datadog does not create a new one. Instead, the values are appended to the existing metric.

EXERCISE – 5.FULLEXAMPLE

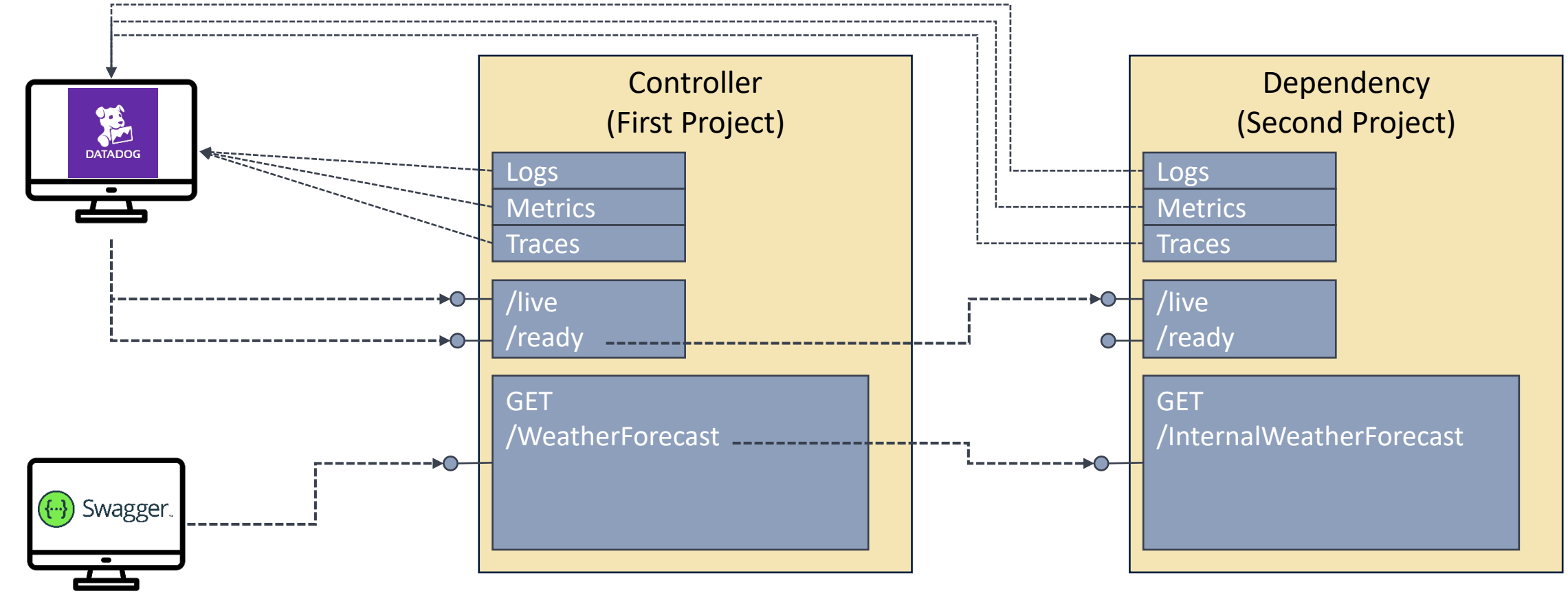
Let’s do an exercise to summarize everything…

- **Two** different .sln **solutions** (to run independently)
  - First project “**Controller**”
    - Is the project that **calls** another one (the second project)
    - Has the Swagger activated (so you can trigger its endpoints - https://localhost:7094/swagger)
    - Has the Health Checks configured to call in cascade the health checks of its dependencies (the second project)
      - Has a UI to observe the Health Checks (https://localhost:7094/healthchecks-ui)
    - Has the logs, metrics, traces enabled and with the **output in the console** (so you can observe also without a Monitoring Tool)
  - Second project “**Dependency**”
    - IS the project **called**
    - Has the Swagger disactivated (no need to have a UI to call its endpoints)
    - Has the Health Checks enabled and will be called by the first project
    - Has the logs, metrics, traces enabled and with the **output in the console** (so you can observe also without a Monitoring Tool)

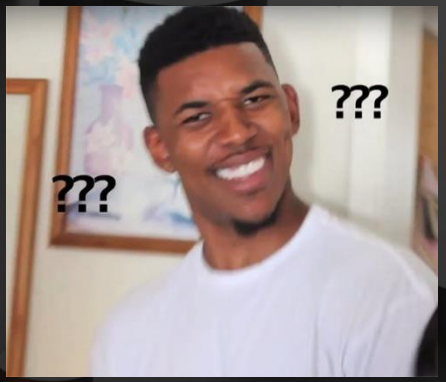
EXERCISE – 5.FULLEXAMPLE – OPENTELEMETRY + CONSOLE



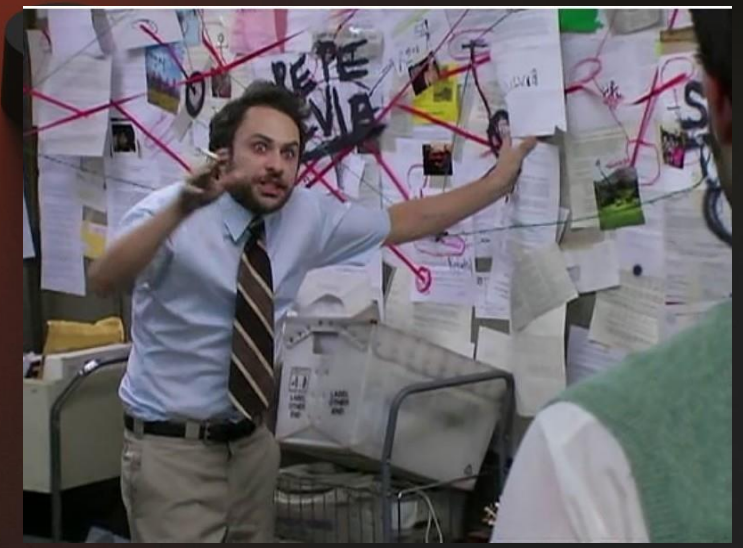
EXERCISE – 5.FULLEXAMPLE - DATADOG



msc.com



Q&A
Time



ENTERPRISE ARCHITECTURE (EA) TEAM